

Dynamic Memory Allocation

COMP211 Connor McMahon

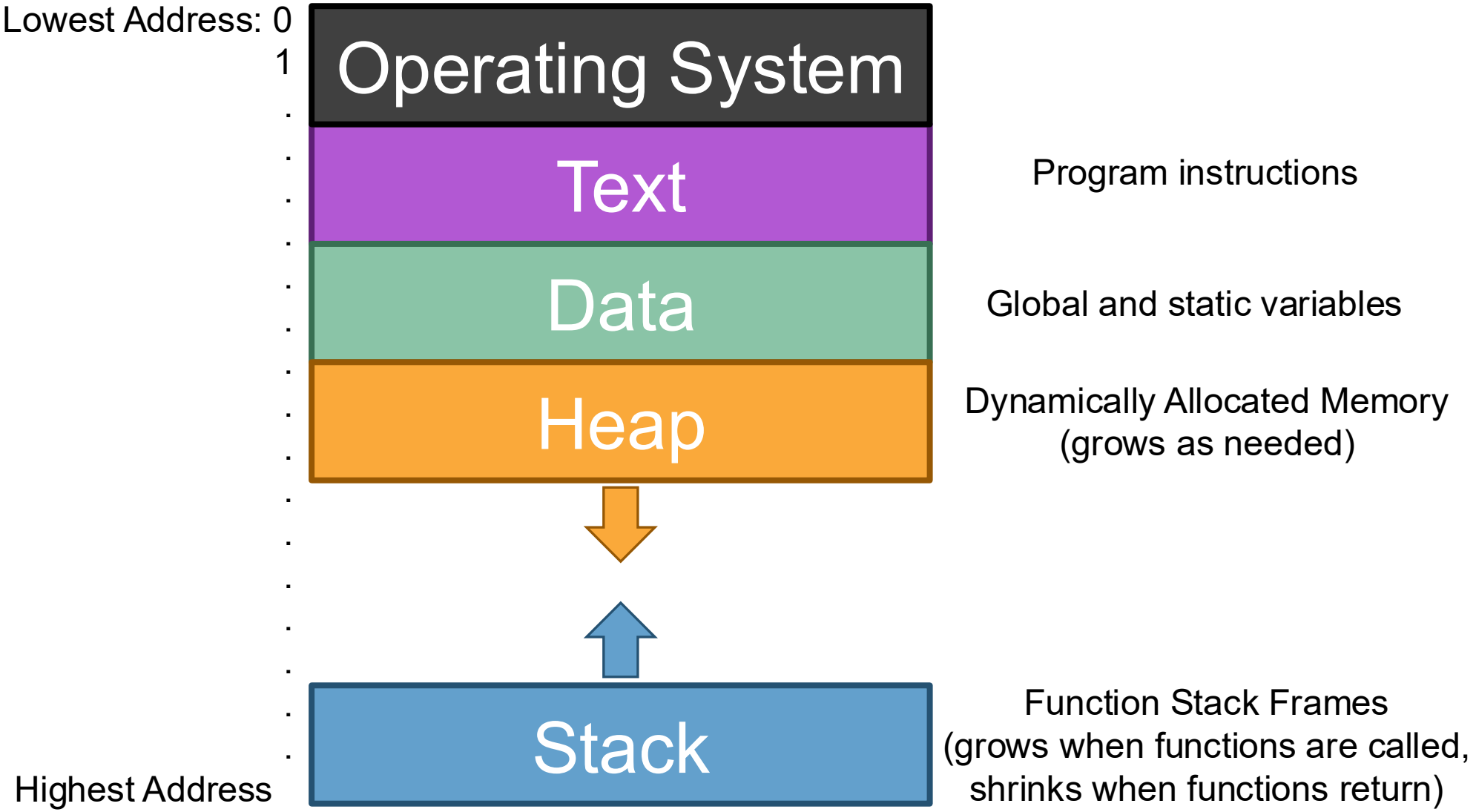
Announcements

- Quiz 2 is next week!
  - You can find all information on Canvas
  - https://uncch.instructure.com/courses/120642/pages/quiz-2

- Don’t forget to fill out the mid-semester feedback form!
  - https://uncch.instructure.com/courses/120642/discussion_topics/893118

Parts of Program Memory

Parts of Program Memory



Compile Time vs. Run Time

- **Compile time**
  - The period when your program's source code is being translated into machine code by the compiler.
  - Happens before execution.
  - The compiler checks syntax, types, and generates the executable file.

Compile Time vs. Run Time

- **Compile time**
  - The period when your program's source code is being translated into machine code by the compiler.
  - Happens before execution.
  - The compiler checks syntax, types, and generates the executable file.

- **Run Time**
  - Run time is the period when your compiled program is executing.
  - Managed by the runtime environment and the operating system.
  - Errors like division by zero, invalid memory access, or file not found happen at run time.

Dynamic Memory Allocation

- Sometimes, the amount of memory needed for a program is not known in advance (at **compile time**)

- **Dynamic memory allocation** lets programs request and release memory while it is running (at **runtime**)

Heap

Stack vs. Heap

- **Stack**
  - Automatically managed
  - Stores function stack frames
  - Memory is freed when a function returns

- **Heap**
  - Manually managed by the programmer
  - Used for dynamically allocated memory
  - Memory remains allocated until the programmer explicitly frees it

Lifetime

- A **lifetime** in C is the period between when memory is allocated and when it is deallocated.

Type	How it works	Duration
Automatic	Local variables inside a function.	Created at the {, destroyed at the }.
Static	Variables declared with static or globals.	They live for the entire run of the program.
Dynamic	Memory allocated via malloc() or calloc().	It lives until you manually call free().

The Three Types of Lifetimes in C

Scope vs. Lifetime

- **Scope**
  - Scope is a **compile-time** concept.
  - The region of your code where a variable's name is valid
  - Defined by curly braces { }.

- **Lifetime**
  - Lifetime is a **run-time** concept.
  - The actual span of time that the variable occupies a physical spot in your computer's memory.
  - Defined by where the variable is stored in memory.

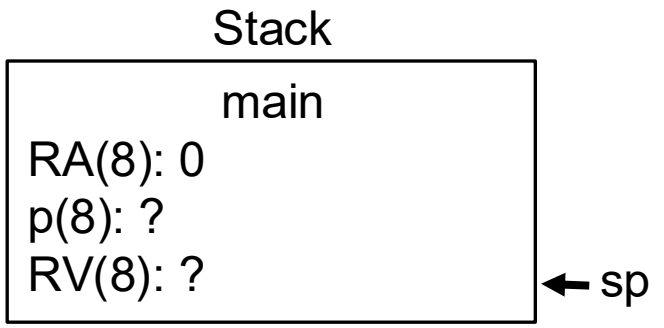
Problem: Dangling Pointers

- **Dangling Pointer:** when the *pointer* lives longer than the *data* it points to.
- Example: Returning a local variable
  - In the example below, x exists on the stack. Once `get_number` finishes, that part of the stack is reclaimed by the system. The pointer p is now pointing at "garbage" memory.

```
1  int32_t* get_number() {
2      int32_t x = 42; // 'x' has Automatic Lifetime
3      return &x; // ERROR: x dies when the function ends!
4  }
5
6
7  int main() {
8      int32_t* p = get_number();
9      printf("%d", *p); // UNDEFINED BEHAVIOR:
10 }
```

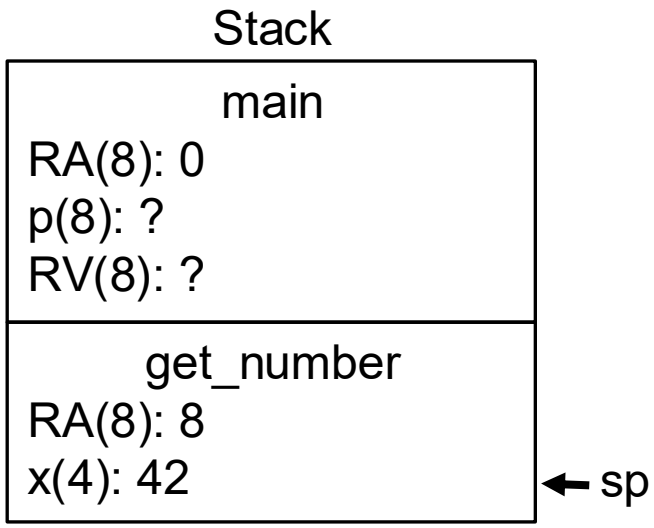
Problem: Dangling Pointers

```
1  int32_t* get_number() {
2      int32_t x = 42; // 'x' has Automatic Lifetime
3      return &x; // ERROR: x dies when the function ends!
4  }
5
6
7  int main() {
8      int32_t* p = get_number();
9      printf("%d", *p); // UNDEFINED BEHAVIOR:
10 }
```



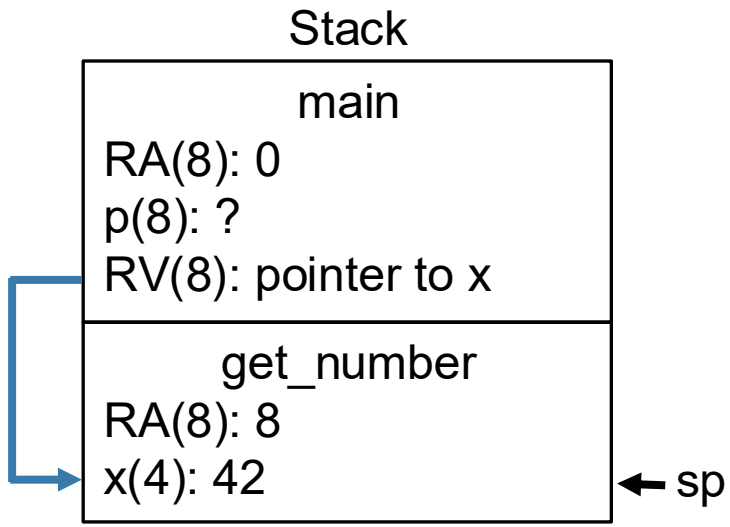
Problem: Dangling Pointers

```
1  int32_t* get_number() {
2      int32_t x = 42; // 'x' has Automatic Lifetime
3      return &x; // ERROR: x dies when the function ends!
4  }
5
6
7  int main() {
8      int32_t* p = get_number();
9      printf("%d", *p); // UNDEFINED BEHAVIOR:
10 }
```



Problem: Dangling Pointers

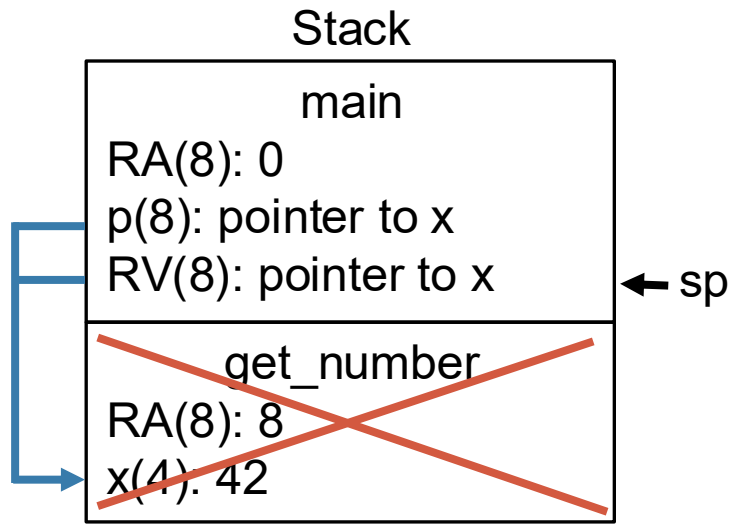
```
1  int32_t* get_number() {
2      int32_t x = 42; // 'x' has Automatic Lifetime
3      return &x; // ERROR: x dies when the function ends!
4  }
5
6
7  int main() {
8      int32_t* p = get_number();
9      printf("%d", *p); // UNDEFINED BEHAVIOR:
10 }
```



Just before get_number returns, it places the return value in main's frame

Problem: Dangling Pointers

```
1  int32_t* get_number() {
2      int32_t x = 42; // 'x' has Automatic Lifetime
3      return &x; // ERROR: x dies when the function ends!
4  }
5
6
7  int main() {
8      int32_t* p = get_number();
9      printf("%d", *p); // UNDEFINED BEHAVIOR:
10 }
```



Now, p is a **dangling pointer**. Once `get_number` finishes, that part of the stack is reclaimed by the system. The pointer p is now pointing at "garbage" memory.

Introducing: `malloc`

- `malloc` is a function defined in the C standard library that allocates a block of memory in the **heap** at runtime.

- `void* malloc(size_t size);`
  - It reserves a block of memory containing `size` bytes.
  - It returns a pointer to the beginning of that block.
  - If the system cannot provide the requested memory, it returns `NULL`.

malloc

- `void *malloc(size_t size);`
- You must include:
  - `#include <stdlib.h>`
- Return type: `void *`
  - A generic pointer to the allocated memory block
- Parameter: `size_t size`
  - Number of bytes to allocate.
- Return value:
  - Pointer to allocated memory on success.
  - `NULL` if the allocation fails.

malloc

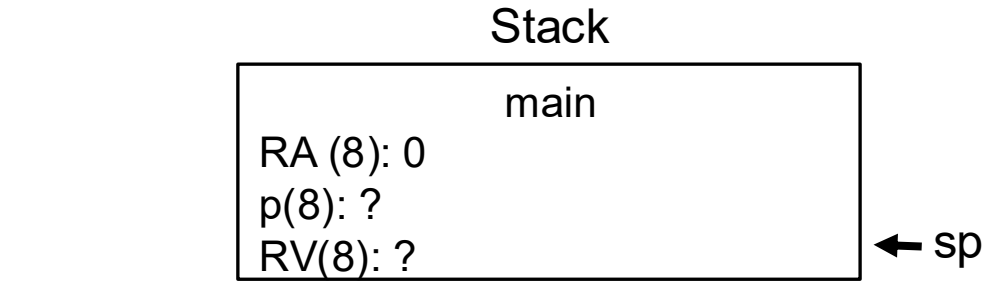
- To fix the previous issue, you must allocate the data on the heap instead of the stack.

```
int* get_number() {
    int* x = malloc(sizeof(int)); // Request heap memory
    *x = 42;
    return x; // This is safe! The memory persists.
}


int main() {
    int* p = get_number();
    printf("%d", *p);
    free(p); // You MUST manually end the lifetime here.
    p = NULL;
}
```

Malloc Example

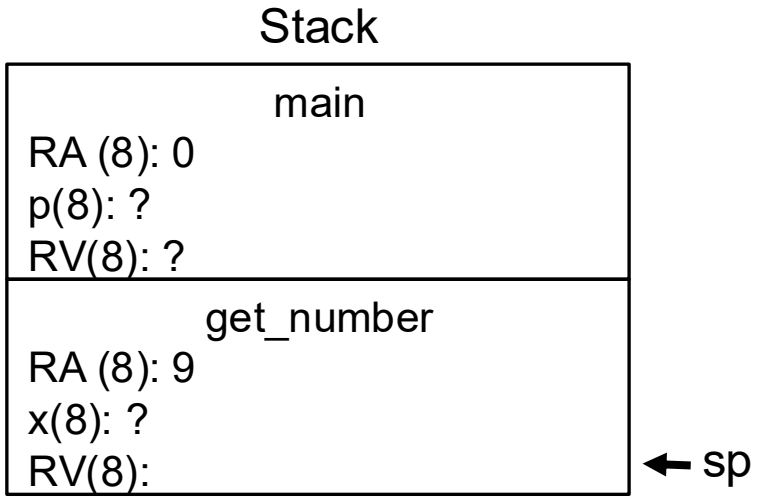
```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



malloc will return an address. All addresses are 8 bytes (64 bits).

Malloc Example

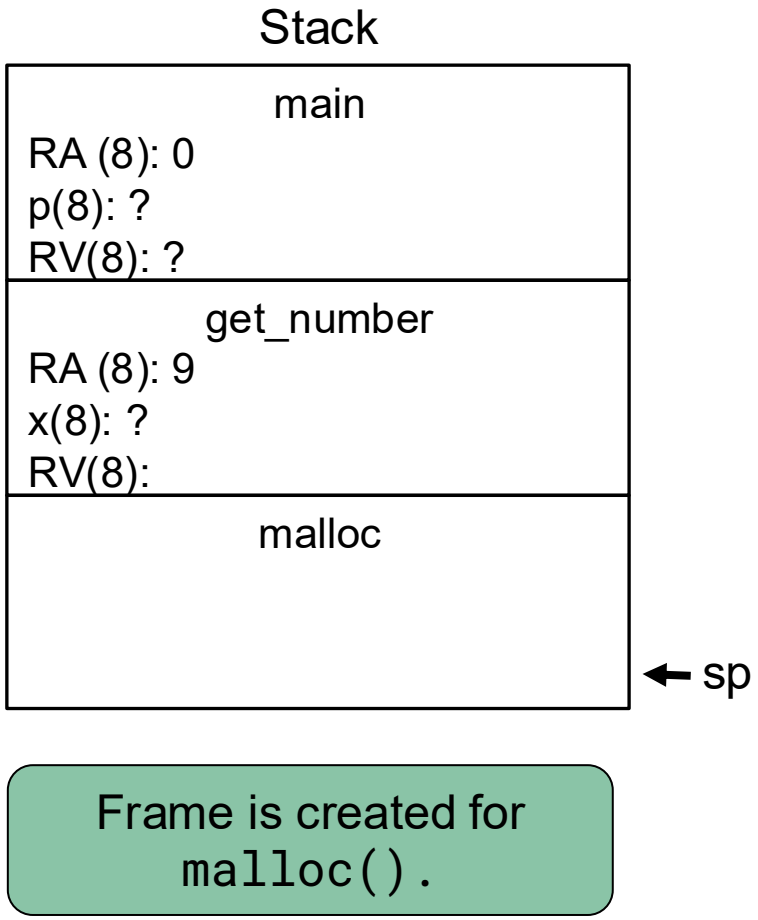
```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



Frame is created for `get_number()`.

Malloc Example

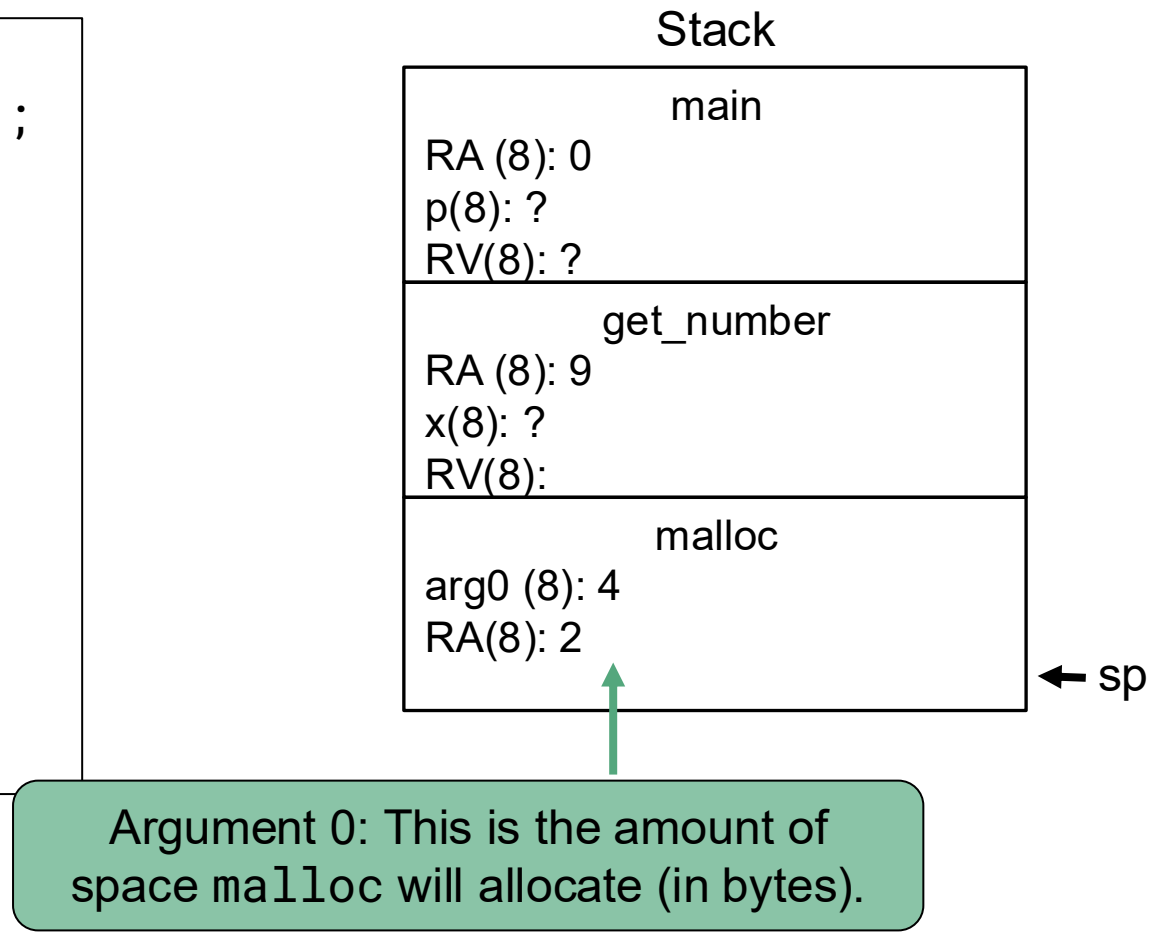
```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



Frame is created for `malloc()`.

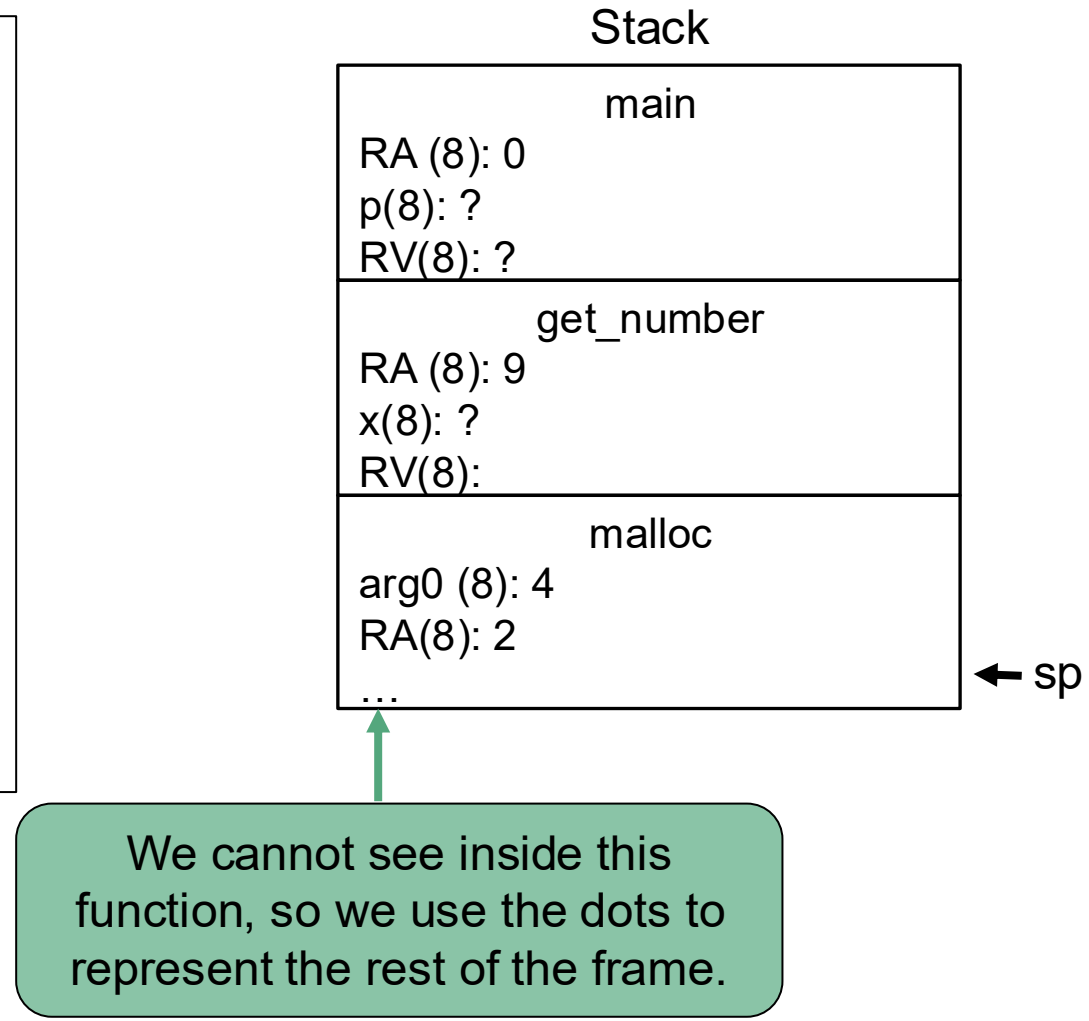
Malloc Example

```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



Malloc Example

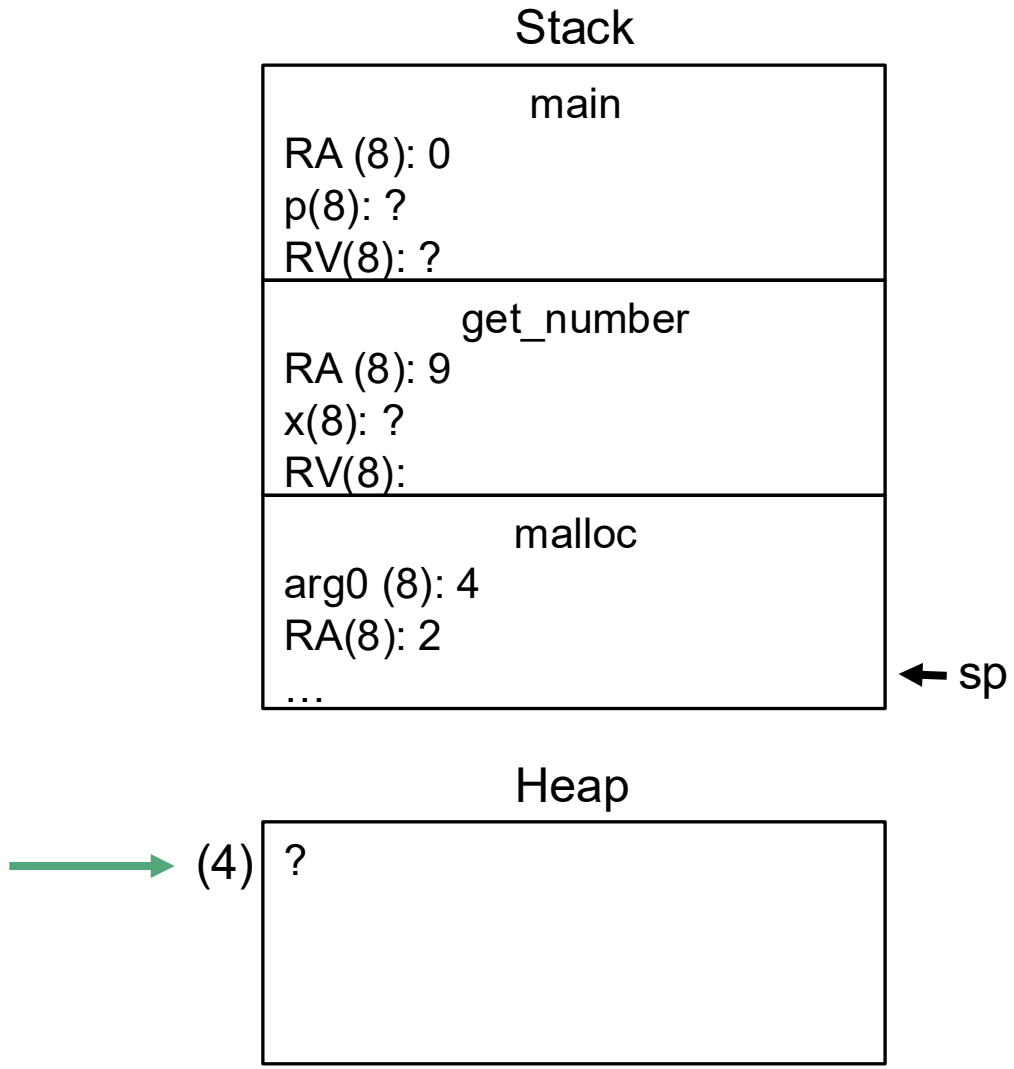
```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



Malloc Example

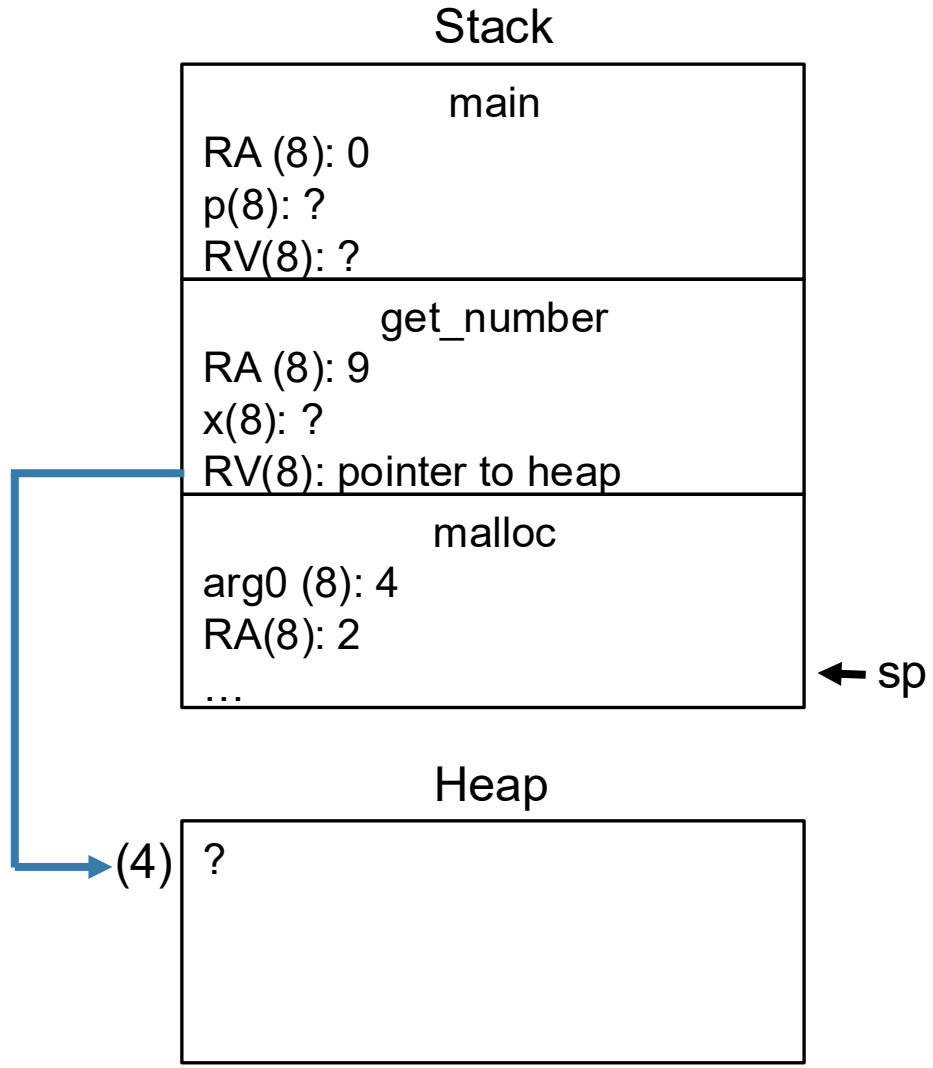
```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```

malloc will allocate 4 bytes on the heap. malloc will give the starting address of this block to get_number().



Malloc Example

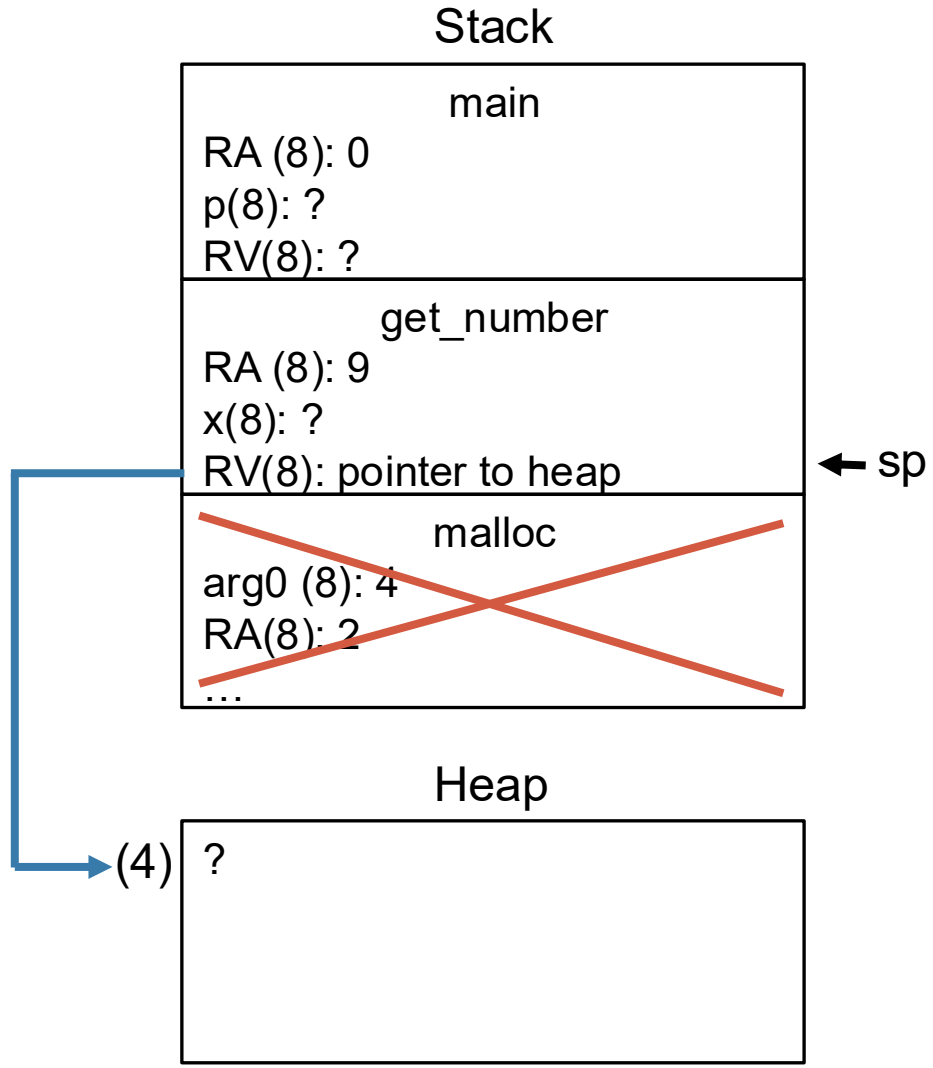
```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



`malloc` places the address in `main`'s frame before returning. The arrow is used to indicate where this variable is pointing to.

Malloc Example

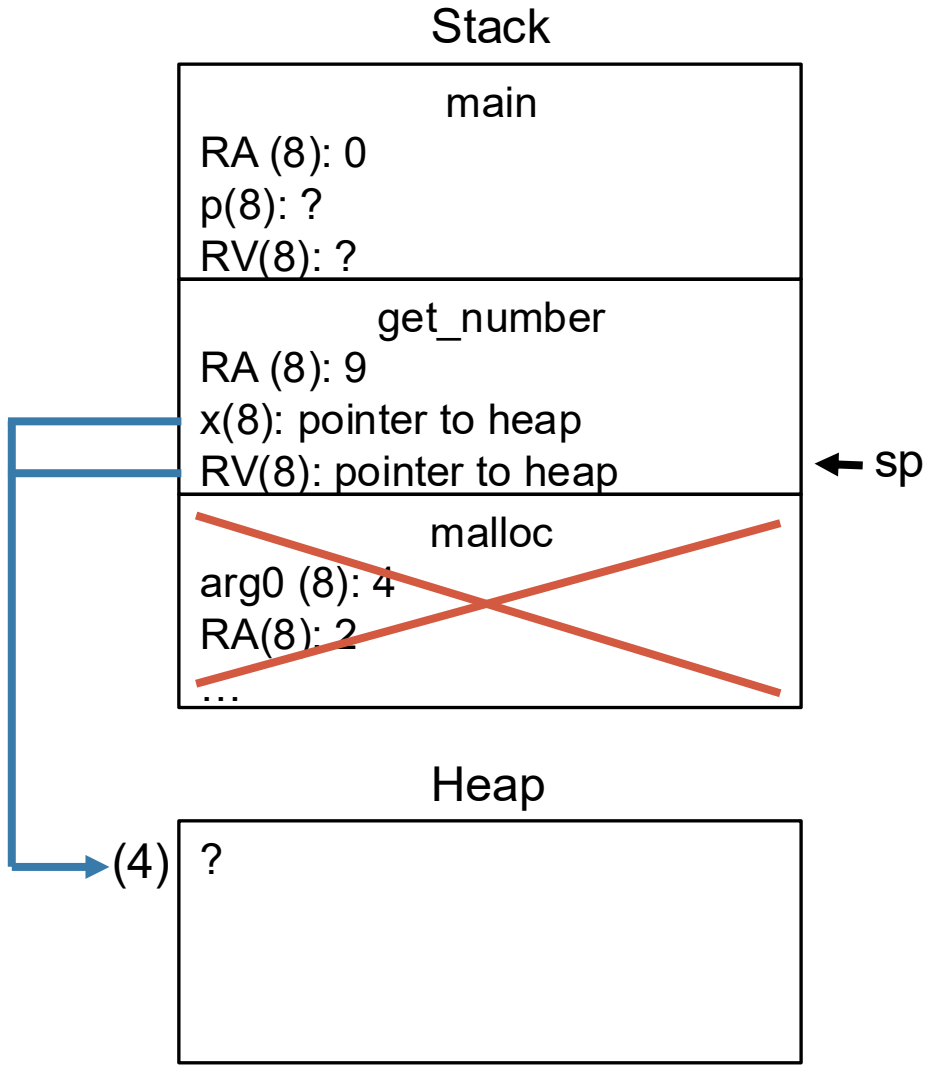
```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



When `malloc` returns, its frame is deallocated.

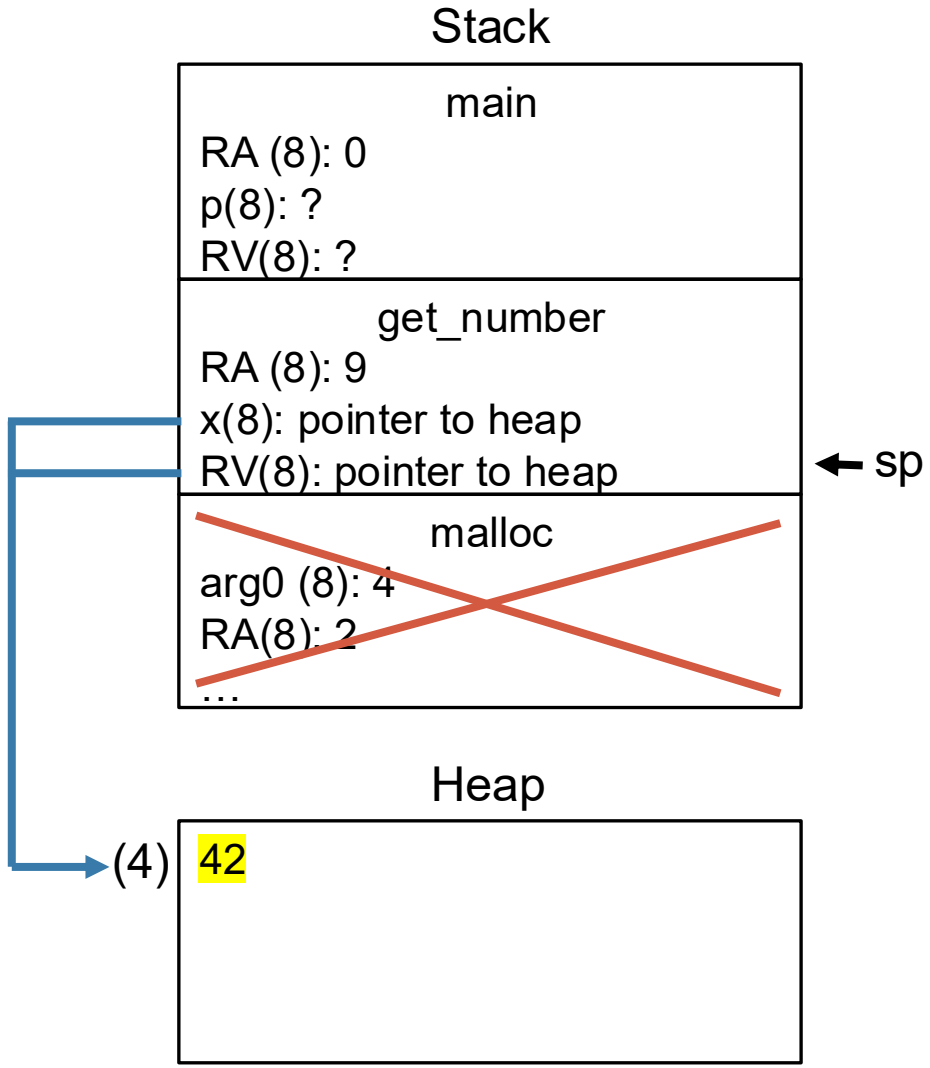
Malloc Example

```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



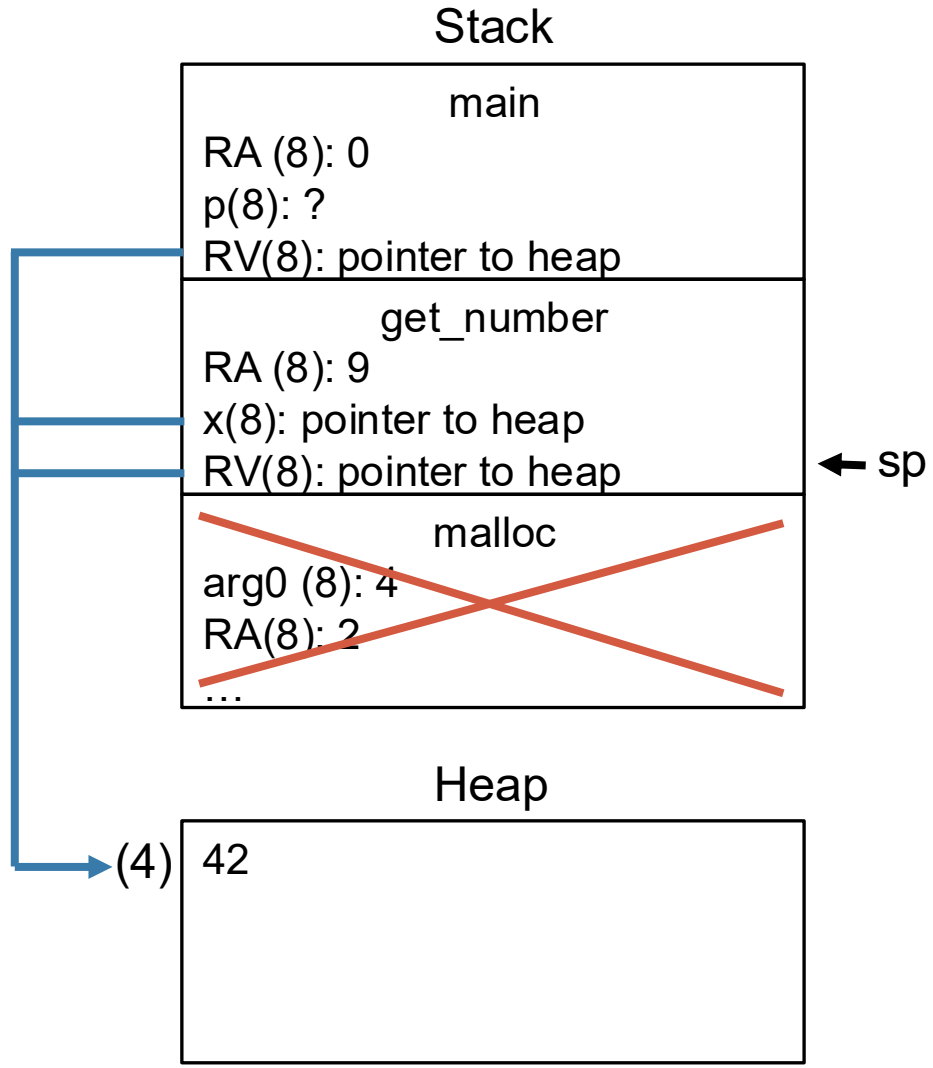
Malloc Example

```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



Malloc Example

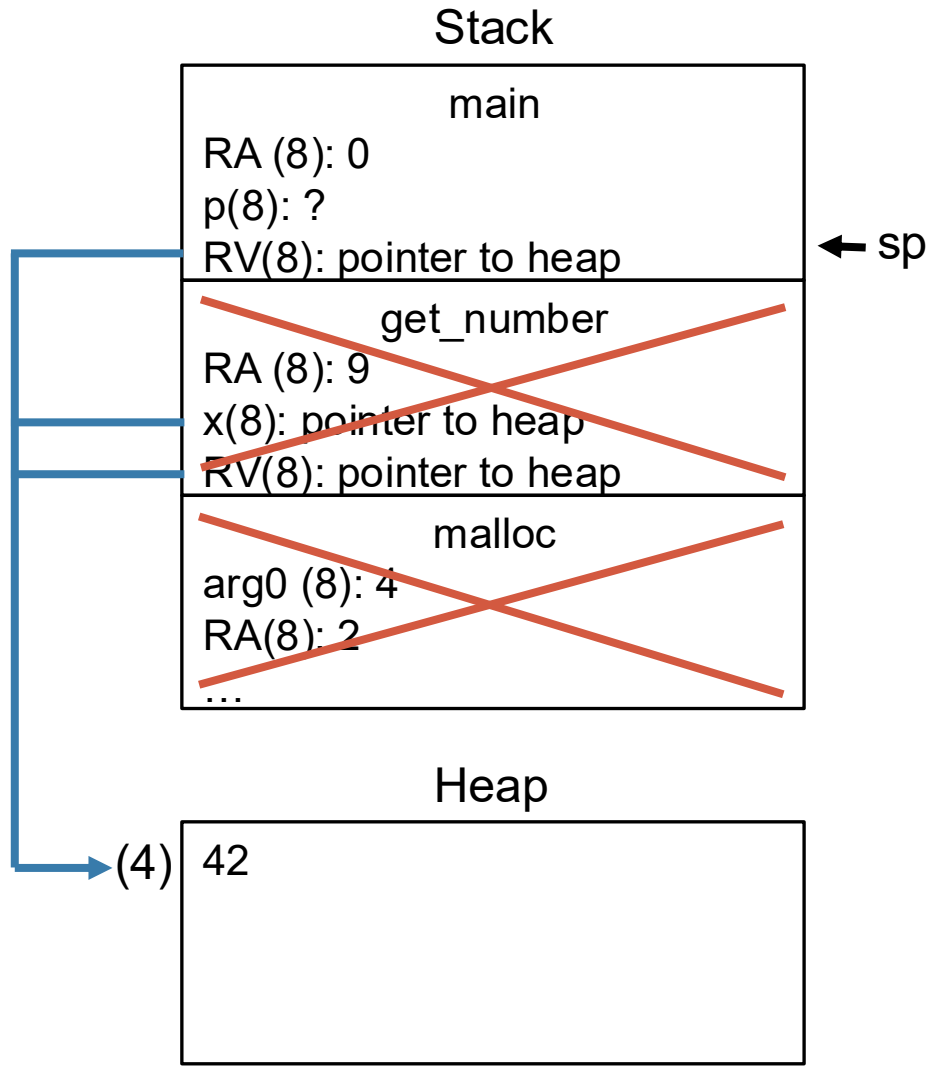
```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



`get_number` places the address in `main`'s frame before returning. The arrow is used to indicate where this variable is pointing to.

Malloc Example

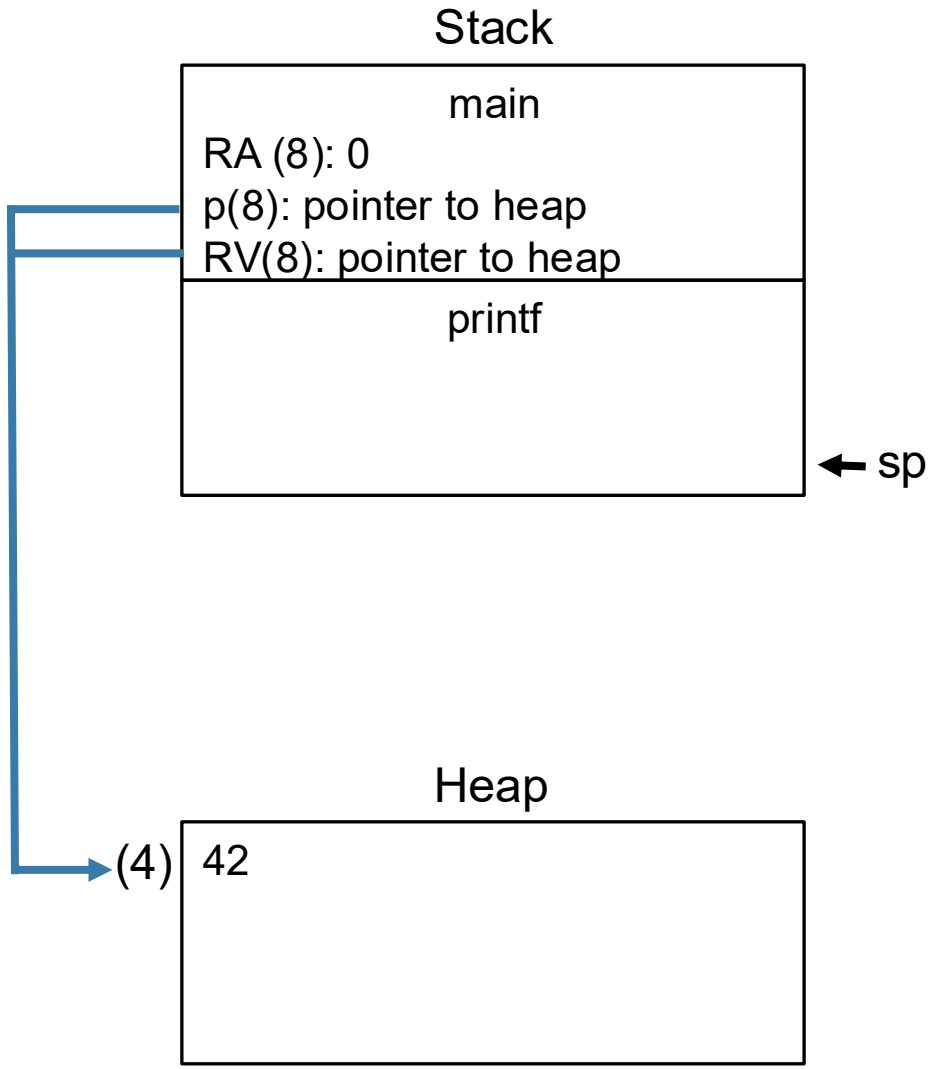
```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



When `get_number` returns, its frame is deallocated.

Malloc Example

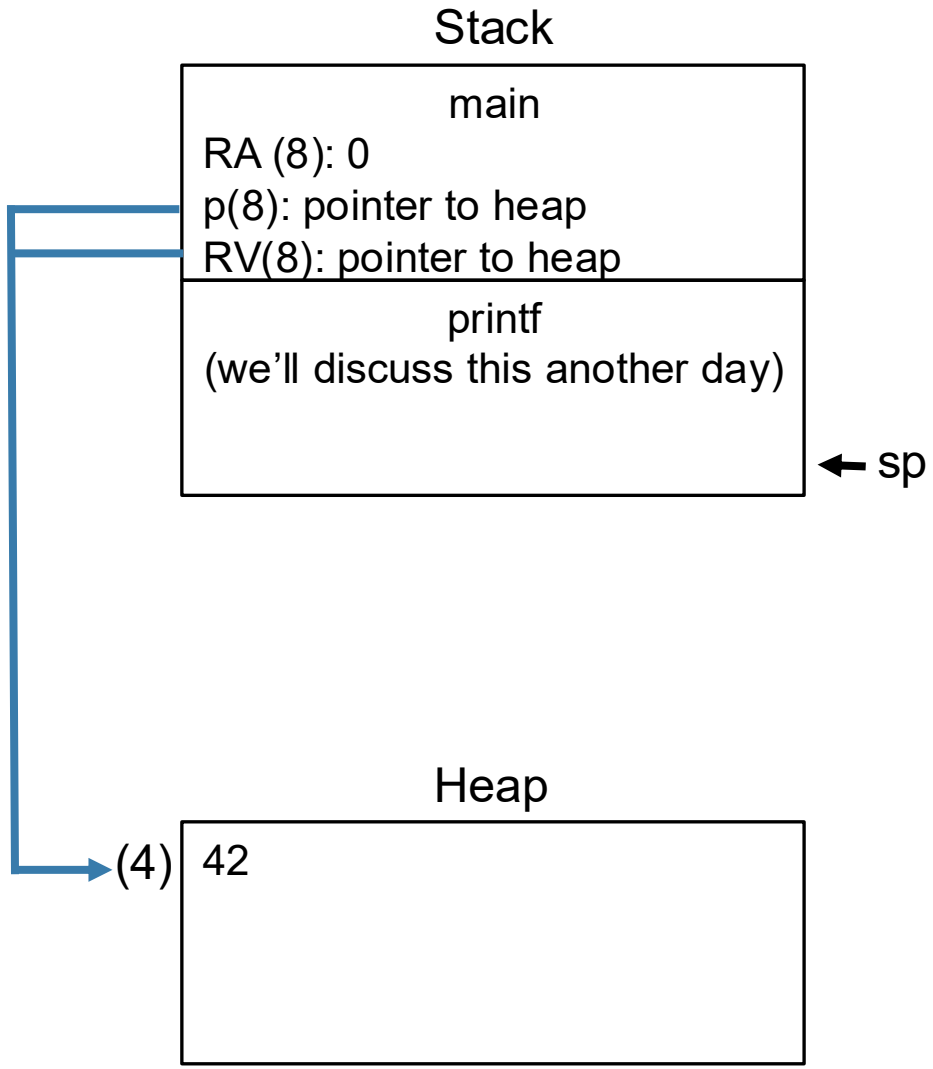
```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



`printf`'s frame will begin at the same location where `get_number`'s frame began.

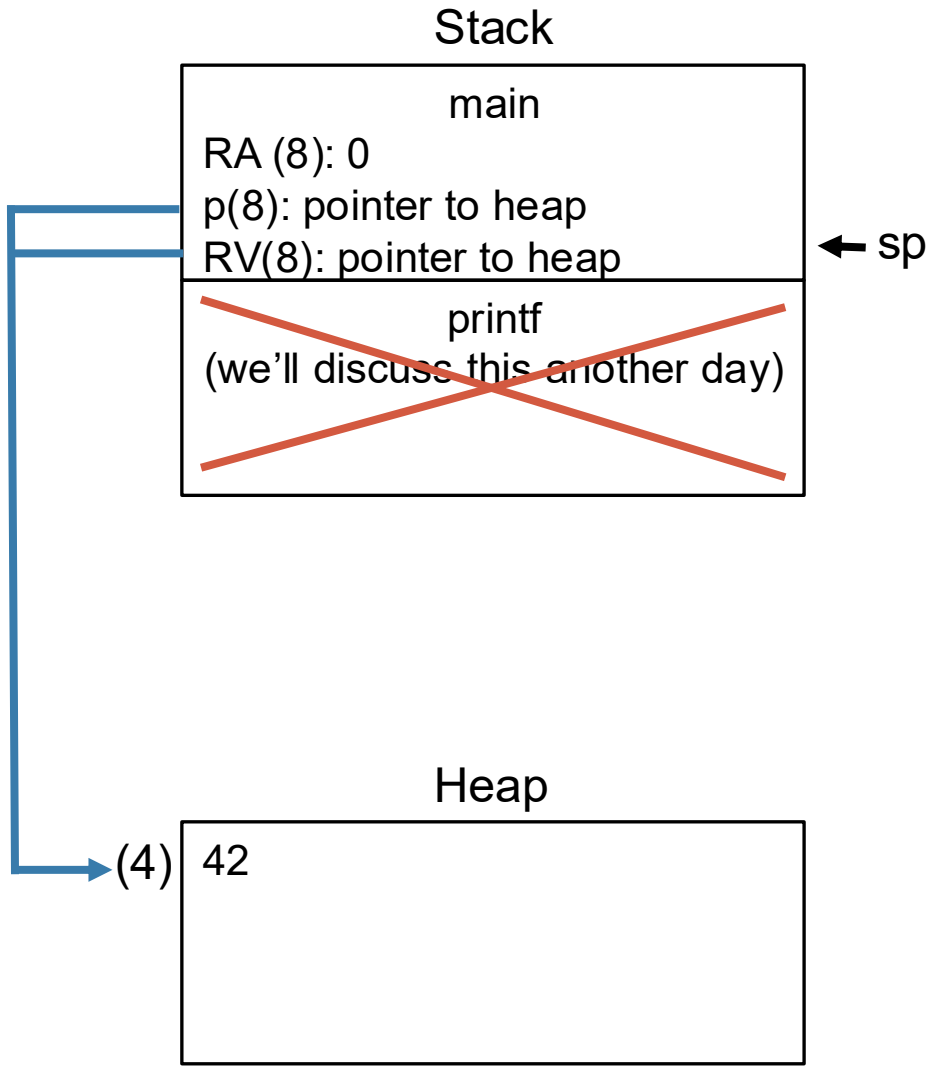
Malloc Example

```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



Malloc Example

```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



When `printf` returns, its frame is deallocated.

Free

- `void free(void *ptr);`

- Releases memory previously allocated by `malloc()`, `calloc()`, or `realloc()`.

- Prevents **memory leaks** by returning the memory to the system.

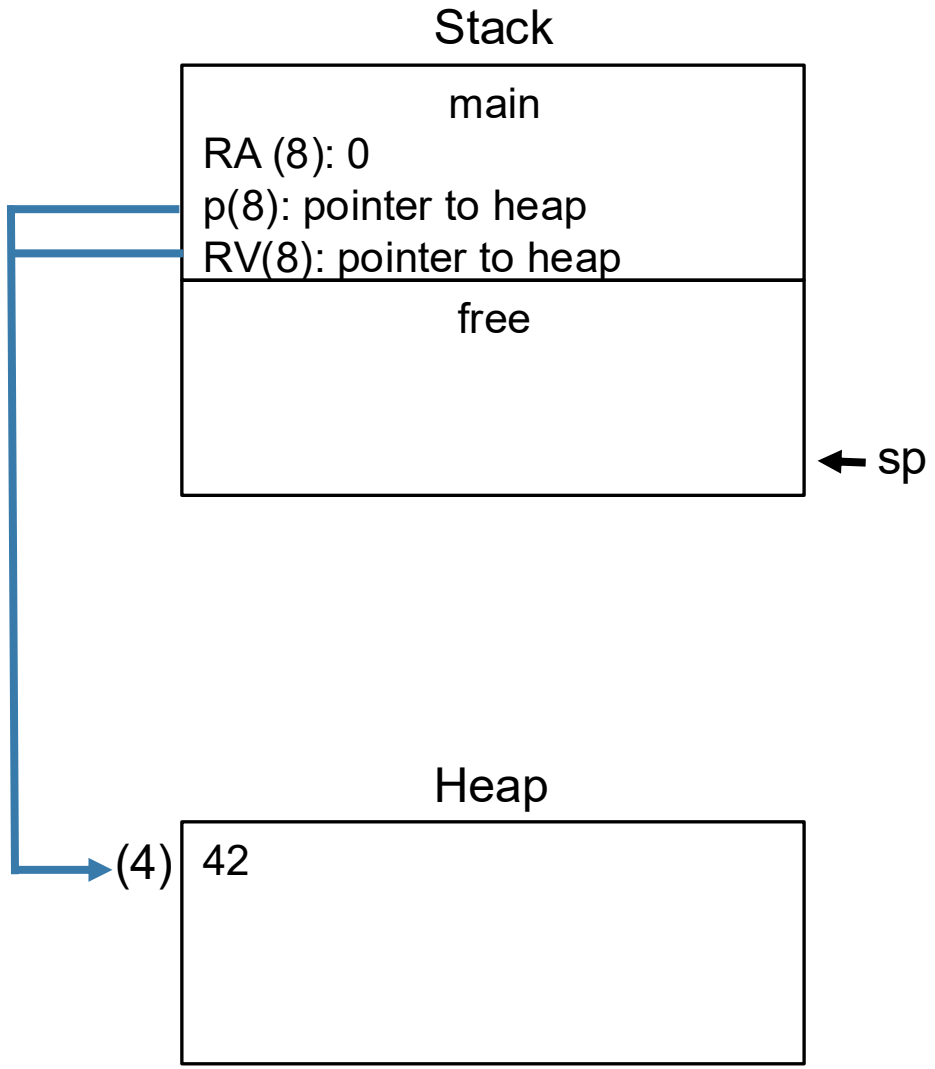
Memory Leak

- A **memory leak** happens when a program allocates memory on the heap but never frees it with `free()` and then loses all references to that allocated memory.

- Why memory leaks are bad
  - The leaked memory remains allocated until the program ends.
  - Repeated leaks cause heap growth -> program uses more and more memory.
  - Over time, this can lead to slowed performance or system crashes due to running out of memory.

Malloc Example

```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```

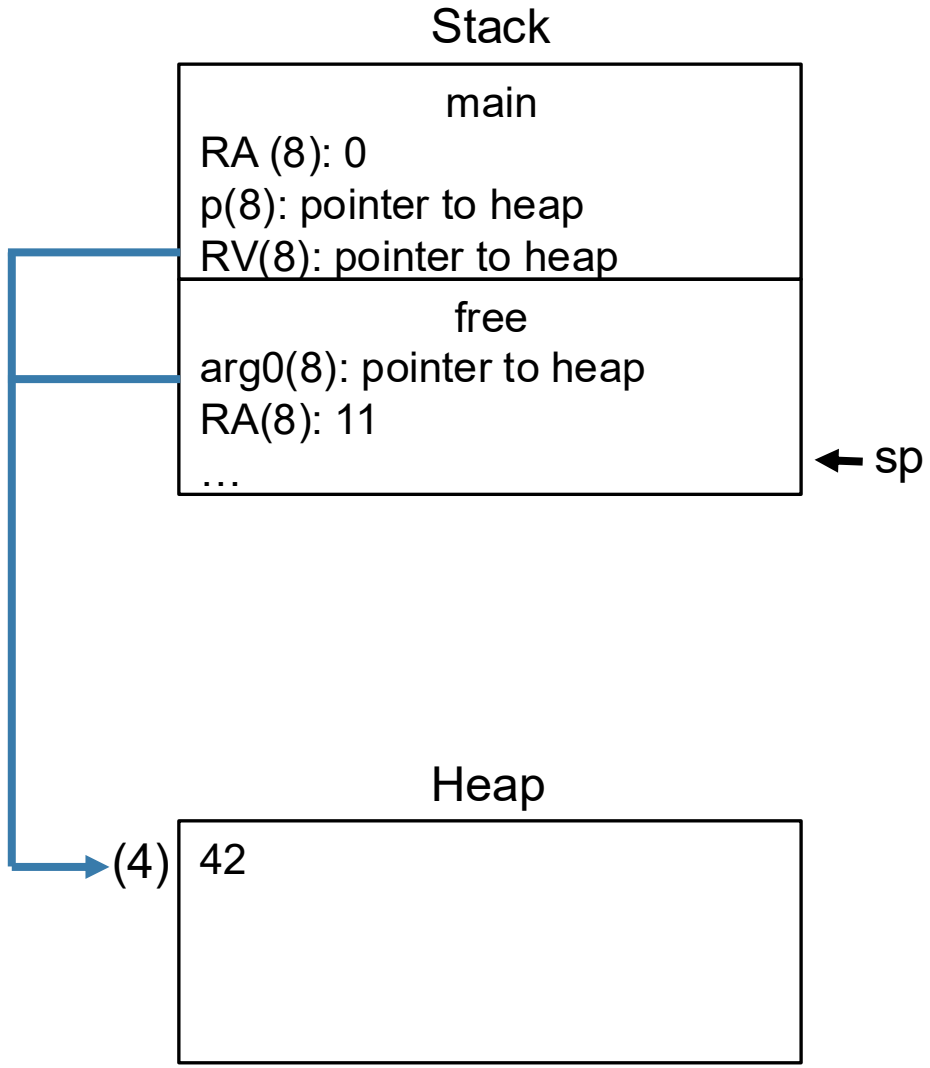


`free`'s frame will begin at the same location where `printf`'s frame began.

Malloc Example

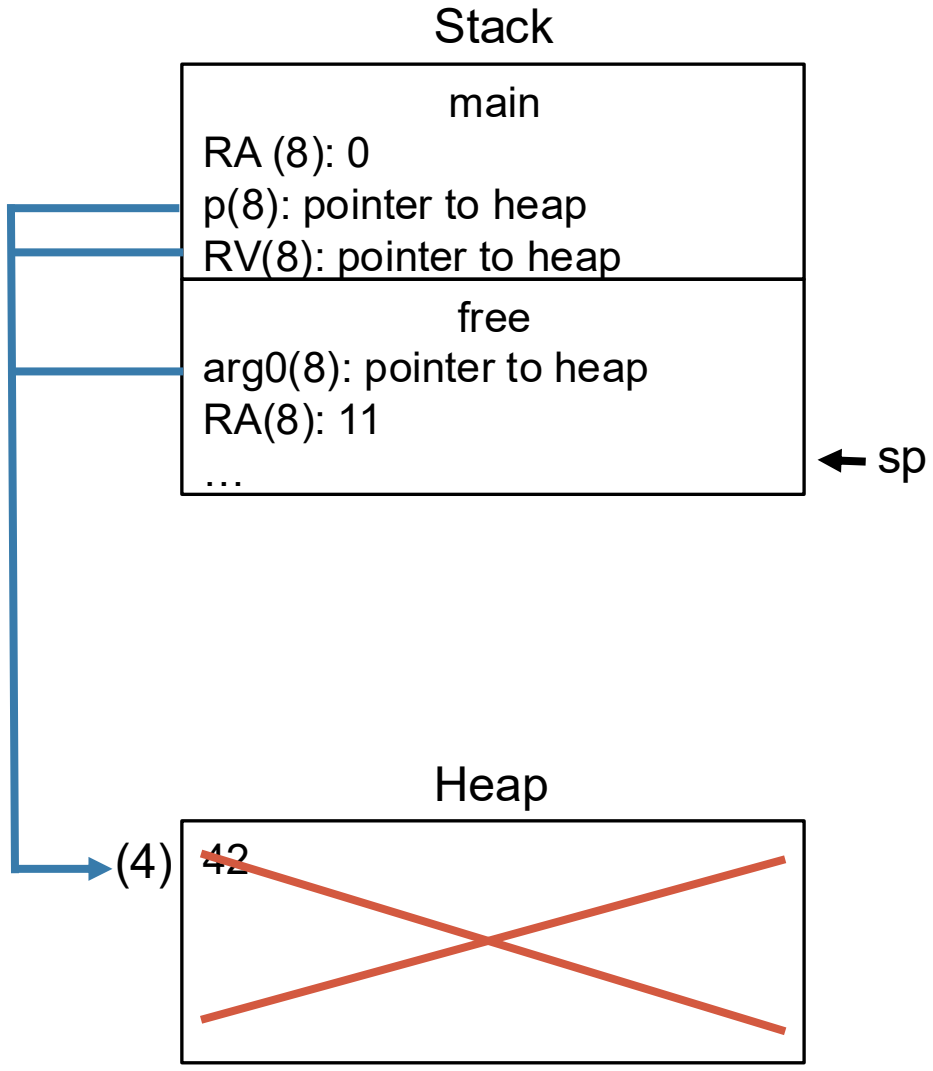
```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```

The `free()` function takes a pointer to the start of the memory block to be released. You don't need to provide the size of the block because the system already knows this from metadata stored in the heap.



Malloc Example

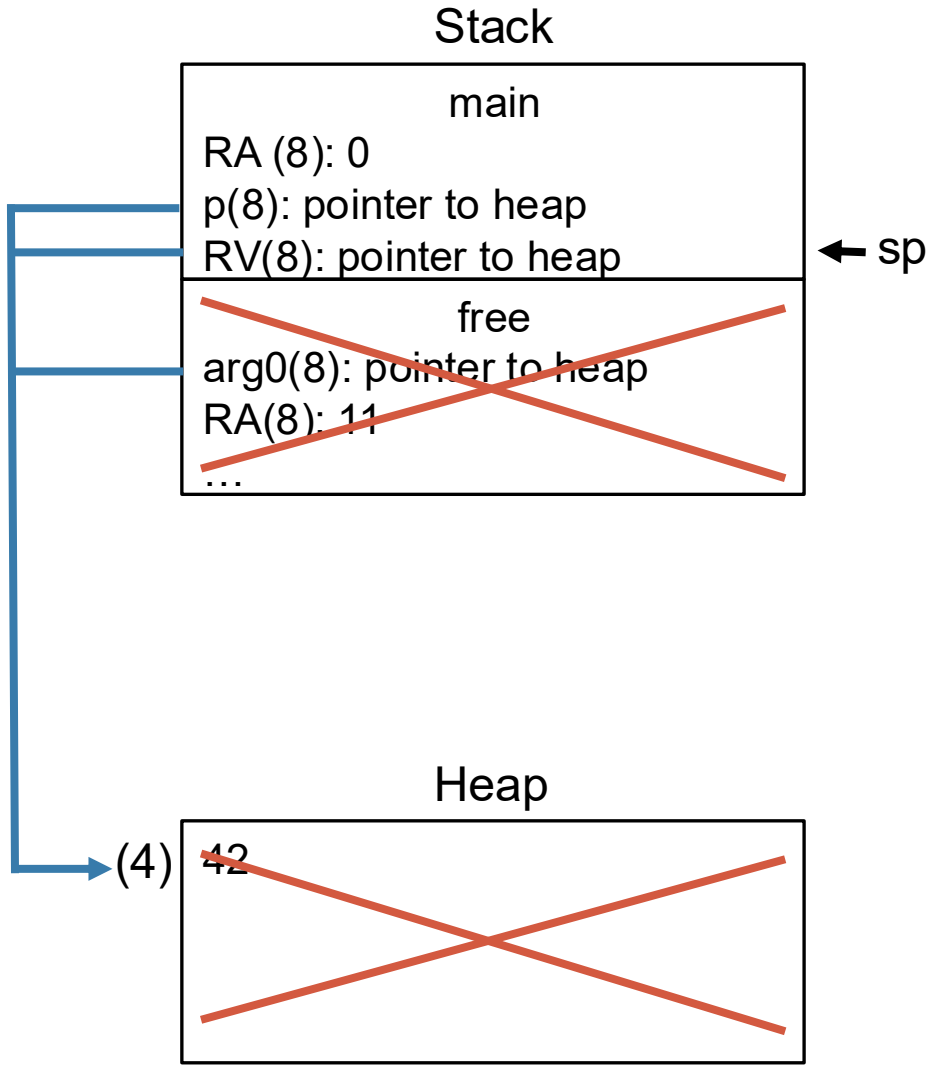
```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



When `free` executes, it deallocates the block in memory.

Malloc Example

```
1  int32_t* get_number() {
2      int32_t* x = malloc(sizeof(int32_t));
3      *x = 42;
4      return x;
5  }
6
7
8  int main() {
9      int32_t* p = get_number();
10     printf("%d", *p);
11     free(p);
12     p = NULL;
13 }
```



When `free` returns, its frame is deallocated.

Free: Important Notes

- Only call `free()` once on each allocated block.
- Never free memory that wasn't allocated with `malloc()`, `calloc()`, or `realloc()`.
- After calling `free()`, the pointer becomes dangling.
  - **Dangling pointer**: a pointer that points to a memory location that is no longer valid
- Freeing memory too early can cause use-after-free bugs.
  - **Use-after-free bugs**: occurs when a program continues to use a pointer after the memory it points to has already been freed.

malloc and free reference

- `void* malloc(size_t size);`
  - It reserves a block of memory containing `size` bytes.
  - It returns a pointer to the beginning of that block.
  - If the system cannot provide the requested memory, it returns `NULL`.

- `void free(void *ptr);`
  - Releases memory previously allocated by `malloc()`, `calloc()`, or `realloc()`.

```c
int* create_even_numbers(int count) {
    // TODO: Allocate an array of 'count' integers on the heap

    int* arr = ________________________________________;

    // Safety check: Ensure malloc succeeded
    if (arr == NULL) return NULL;

    for (int i = 0; i < count; i++) {
        arr[i] = i * 2;
    }

    // TODO: Return the pointer to the heap memory

    return ________;
}

int main() {
    int n = 5;
    int* my_evens = create_even_numbers(n);

    if (my_evens != NULL) {
        printf("The third even number is: %d\n", my_evens[2]);

        // TODO: Deallocate the array to avoid a memory leak


    }
    return 0;
}
```



```c
int* create_even_numbers(int count) {
    // Allocate an array of 'count' integers on the heap
    int* arr = malloc(count * sizeof(int));

    // Safety check: Ensure malloc succeeded
    if (arr == NULL) return NULL;

    for (int i = 0; i < count; i++) {
        arr[i] = i * 2;
    }

    // Return the pointer to the heap memory
    return arr;
}

int main() {
    int n = 5;
    int* my_evens = create_even_numbers(n);

    if (my_evens != NULL) {
        printf("The third even number is: %d\n", my_evens[2]);

        // Deallocate the array to avoid a memory leak
        free(my_evens);
        my_evens = NULL;
    }

    return 0;
}
```

The Unknown Size Array

- The most common use for dynamic memory is when you don't know how much data you'll have until the program is actually running
  - (e.g., asking a user how many items are in their shopping list).

- If we used a normal array like int grades[10];, we'd be stuck with exactly 10. malloc lets the lifetime of the array's **size** be determined at runtime.

- See example on next slide.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("How many grades do you want to enter? ");
    scanf("%d", &n);

    // 1. Allocate memory on the HEAP
    // We request space for 'n' integers
    int* grades = malloc(n * sizeof(int));

    // ALWAYS check if malloc failed (if the computer is out of memory)
    if (grades == NULL) return 1;

    for (int i = 0; i < n; i++) {
        grades[i] = i * 10; // Assigning some values
        printf("Grade %d: %d\n", i, grades[i]);
    }

    // 2. Deallocate the array manually!
    free(grades);
    grades = NULL;

    return 0;
}
```

Resizing Memory

- Sometimes you start with a small array and later need to resize the array to fit more data.

- This is how ArrayList in Java or list in Python works under the hood!

realloc

- `realloc()` resizes previously allocated memory.

- `ptr = realloc(ptr, new_size);`
  - Preserves existing data up to the smaller of the old and new sizes.
  - May move memory to a new location if the current block cannot be extended.
  - Returns `NULL` if reallocation fails (the original pointer is still valid).
  - Commonly used for dynamic arrays that grow or shrink at runtime.

- Declared in `<stdlib.h>`

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Start with space for 2 integers
    int* list = malloc(2 * sizeof(int));
    list[0] = 100;
    list[1] = 200;

    // Oh no, we need room for a 3rd!
    // realloc moves the data to a bigger spot if needed
    int* temp = realloc(list, 3 * sizeof(int));

    if (temp != NULL) {
        list = temp;
        list[2] = 300;
    }

    for (int i = 0; i < 3; i++) printf("%d ", list[i]);

    printf("\n");
    free(list); // Clean up the final version
    list = NULL;
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // We start with space for 2 items
    int* list = malloc(2 * sizeof(int));
    list[0] = 10;
    list[1] = 20;
    printf("Original list: %d, %d\n", list[0], list[1]);

    // TODO: Use realloc to grow the 'list' to hold 4 integers

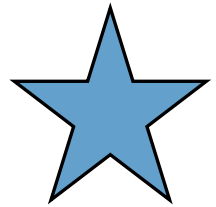
    int* temp = ________________________________________;

    if (temp != NULL) {
        list = temp;
        list[2] = 30;
        list[3] = 40;
        printf("New items: %d, %d\n", list[2], list[3]);
    }

    // TODO: Clean up the memory



    return 0;
}
```



```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // We start with space for 2 items
    int* list = malloc(2 * sizeof(int));

    list[0] = 10;
    list[1] = 20;

    printf("Original list: %d, %d\n", list[0], list[1]);

    // Use realloc to grow the 'list' to hold 4 integers
    int* temp = realloc(list, 4 * sizeof(int));

    if (temp != NULL) {
        list = temp;
        list[2] = 30;
        list[3] = 40;

        printf("New items: %d, %d\n", list[2], list[3]);
    }

    // Clean up the memory
    free(list);
    list = NULL;

    return 0;
}
```

High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```

Step 1: Allocate space for everything that will be stored in `main`'s frame.

High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

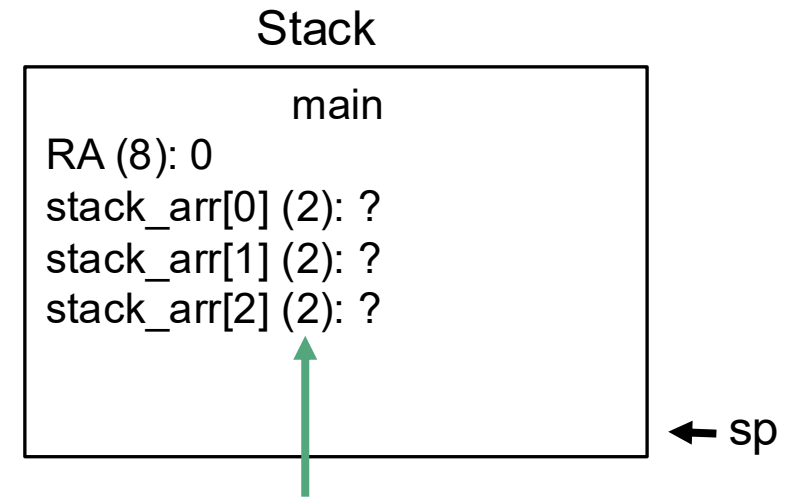
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



Step 1: Allocate space for everything that will be stored in `main`'s frame.

High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

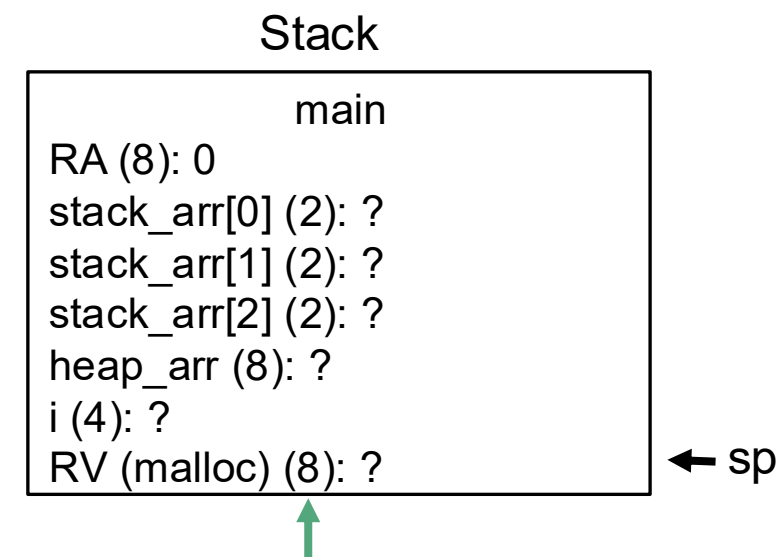
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

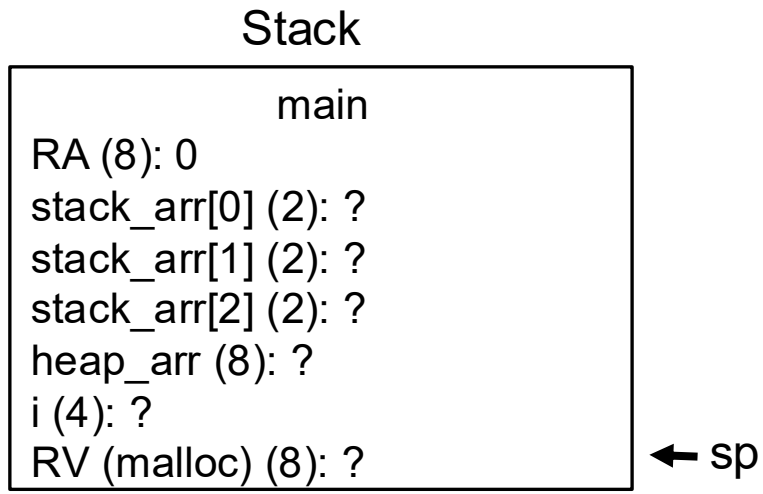
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

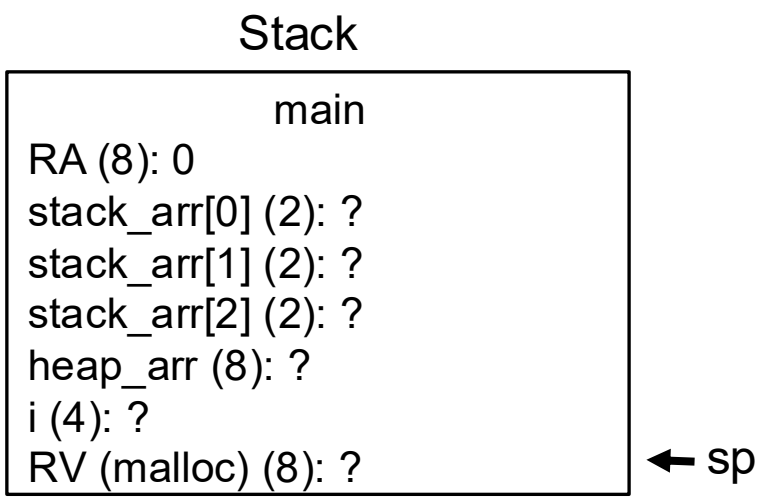
    return 0;
}
```



Step 2: Walk through program line-by-line.

High-Level Memory Diagram

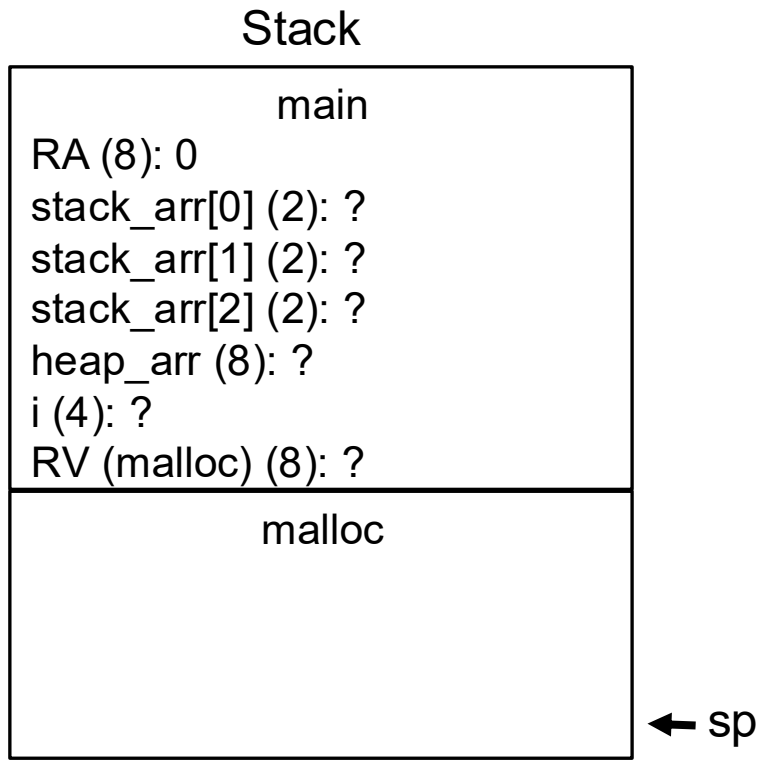
```
1  #include <ctype.h>
2  #include <stdlib.h>
3
4  #define ARRAY_SIZE 3
5
6  int main() {
7      int16_t stack_arr[ARRAY_SIZE];
8      int16_t* heap_arr =
           (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));
9
10     if (heap_arr == NULL) {
11         printf("Memory allocation failed!\n");
12         return 1;
13     }
14
15
16     for (int32_t i = 0; i < ARRAY_SIZE; i++) {
17         stack_arr[i] = i;
18         heap_arr[i] = i * 2;
19     }
20
21     free(heap_arr);
22     heap_arr = NULL;
23
24     return 0;
25 }
```



Nothing changes here.

High-Level Memory Diagram

```
1  #include <ctype.h>
2  #include <stdlib.h>
3
4  #define ARRAY_SIZE 3
5
6  int main() {
7      int16_t stack_arr[ARRAY_SIZE];
8      int16_t* heap_arr =
           (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));
9
10     if (heap_arr == NULL) {
11         printf("Memory allocation failed!\n");
12         return 1;
13     }
14
15
16     for (int32_t i = 0; i < ARRAY_SIZE; i++) {
17         stack_arr[i] = i;
18         heap_arr[i] = i * 2;
19     }
20
21     free(heap_arr);
22
23     return 0;
24 }
25
```



Next, we will call `malloc`. We need to make a frame for it.

High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

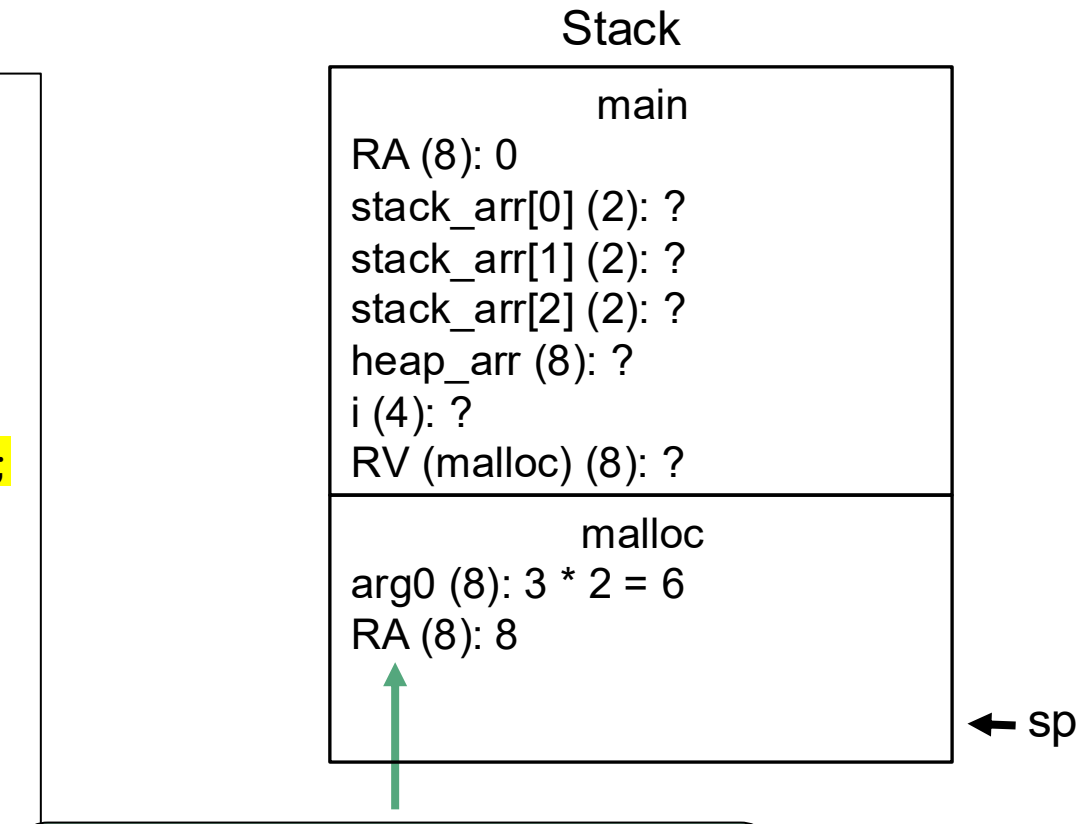
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



Argument 0: This is the amount of space `malloc` will allocate (in bytes).

High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

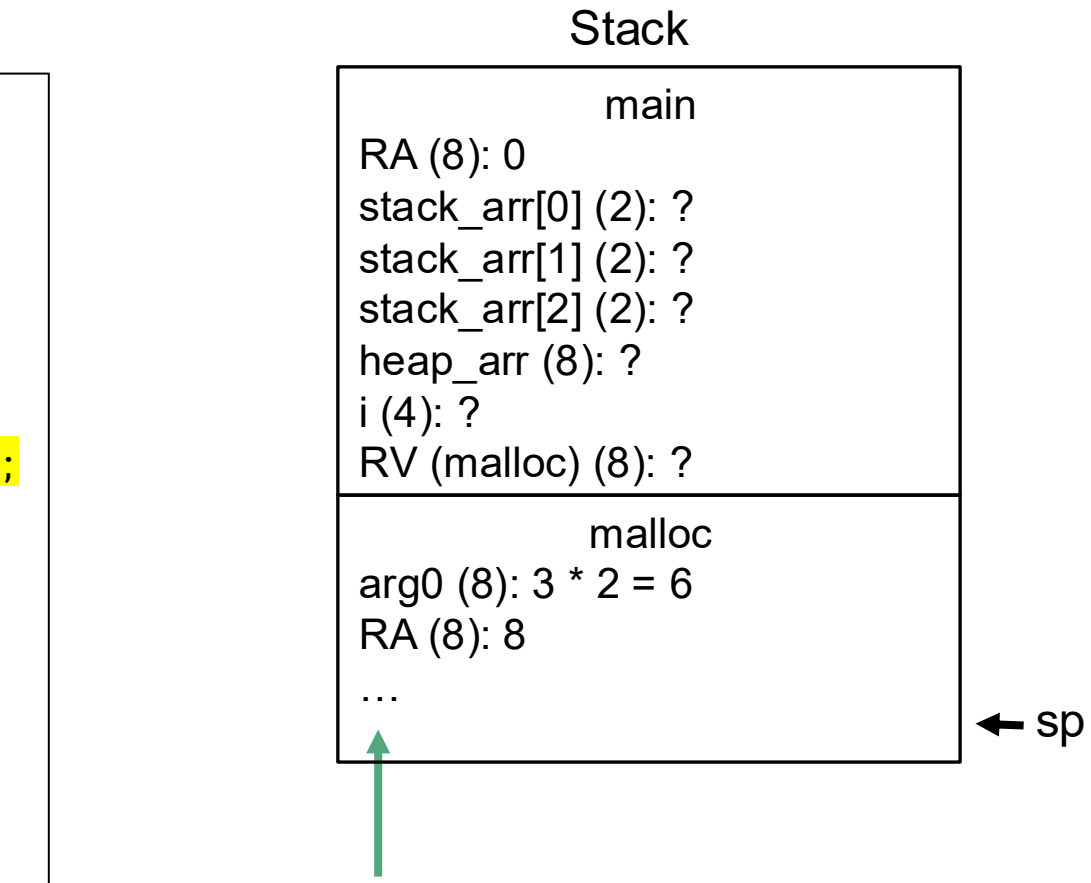
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



We cannot see inside this function, so we use the dots to represent the rest of the frame.

High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

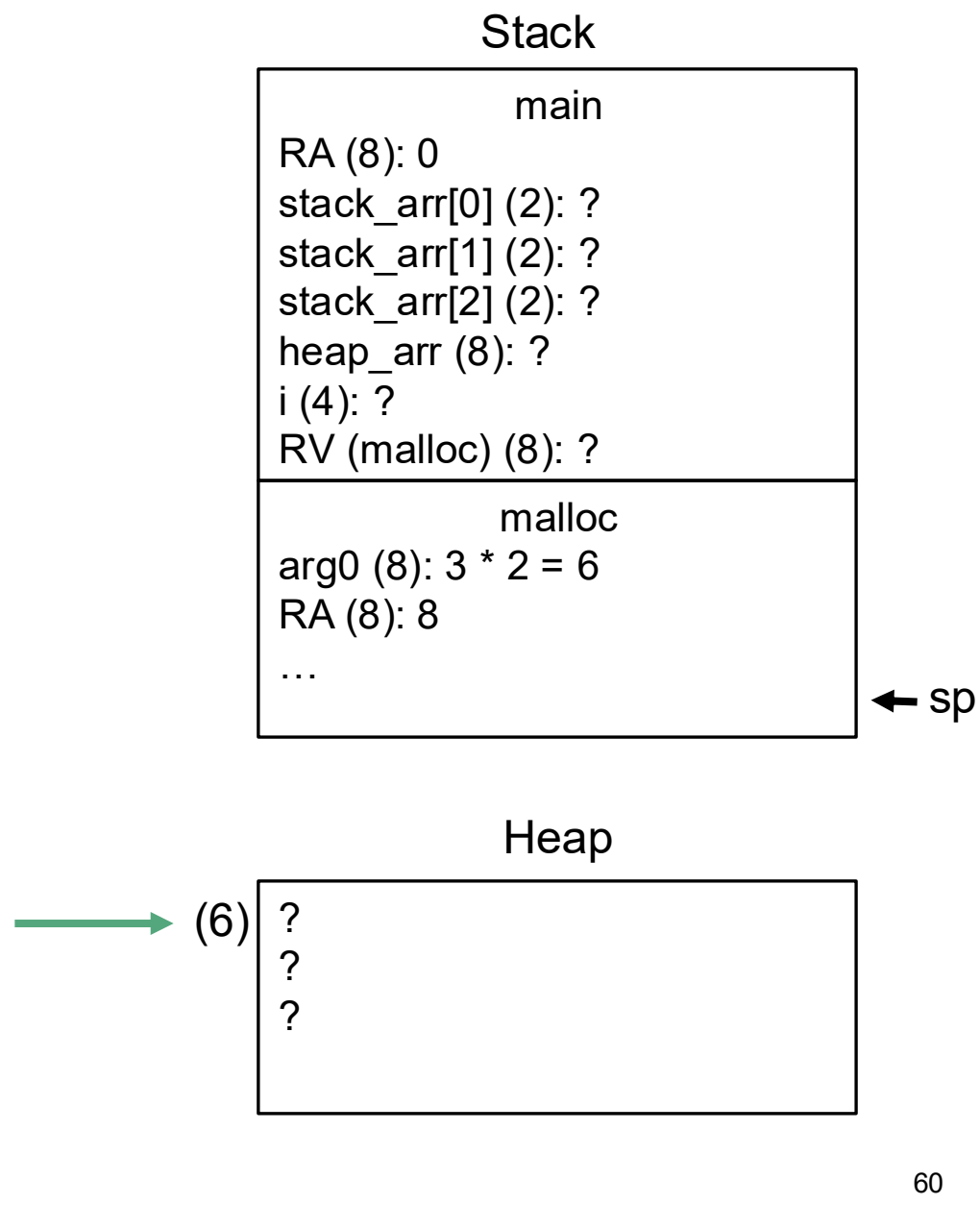
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

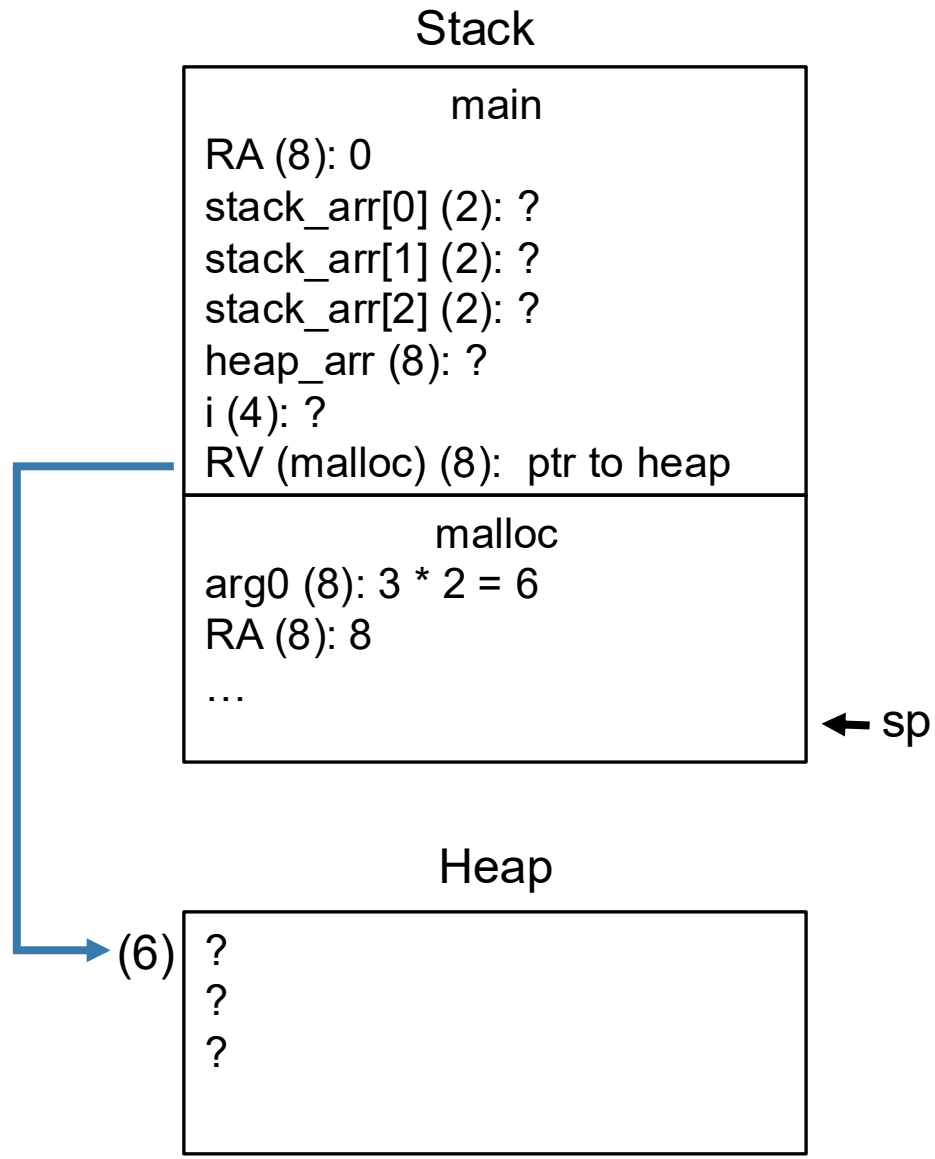
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);

    return 0;
}
```

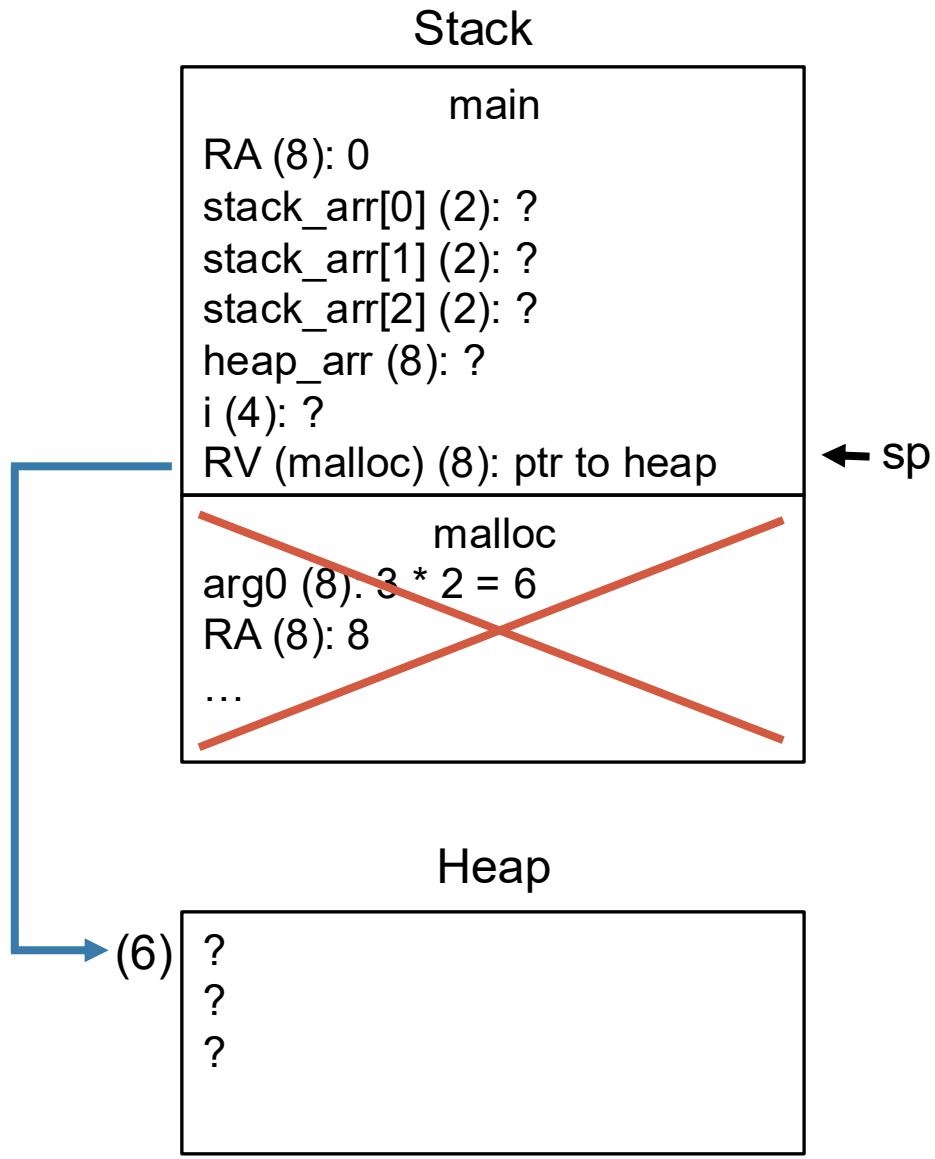


`malloc` places the address in `main`'s frame before returning. The arrow is used to indicate where this variable is pointing to.

High-Level Memory Diagram

```
1  #include <ctype.h>
2  #include <stdlib.h>
3
4  #define ARRAY_SIZE 3
5
6  int main() {
7      int16_t stack_arr[ARRAY_SIZE];
8      int16_t* heap_arr =
           (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));
9
10     if (heap_arr == NULL) {
11         printf("Memory allocation failed!\n");
12         return 1;
13     }
14
15
16     for (int32_t i = 0; i < ARRAY_SIZE; i++) {
17         stack_arr[i] = i;
18         heap_arr[i] = i * 2;
19     }
20
21     free(heap_arr);
22     heap_arr = NULL;
23
24     return 0;
25 }
```

When malloc returns, its frame is deallocated.



High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

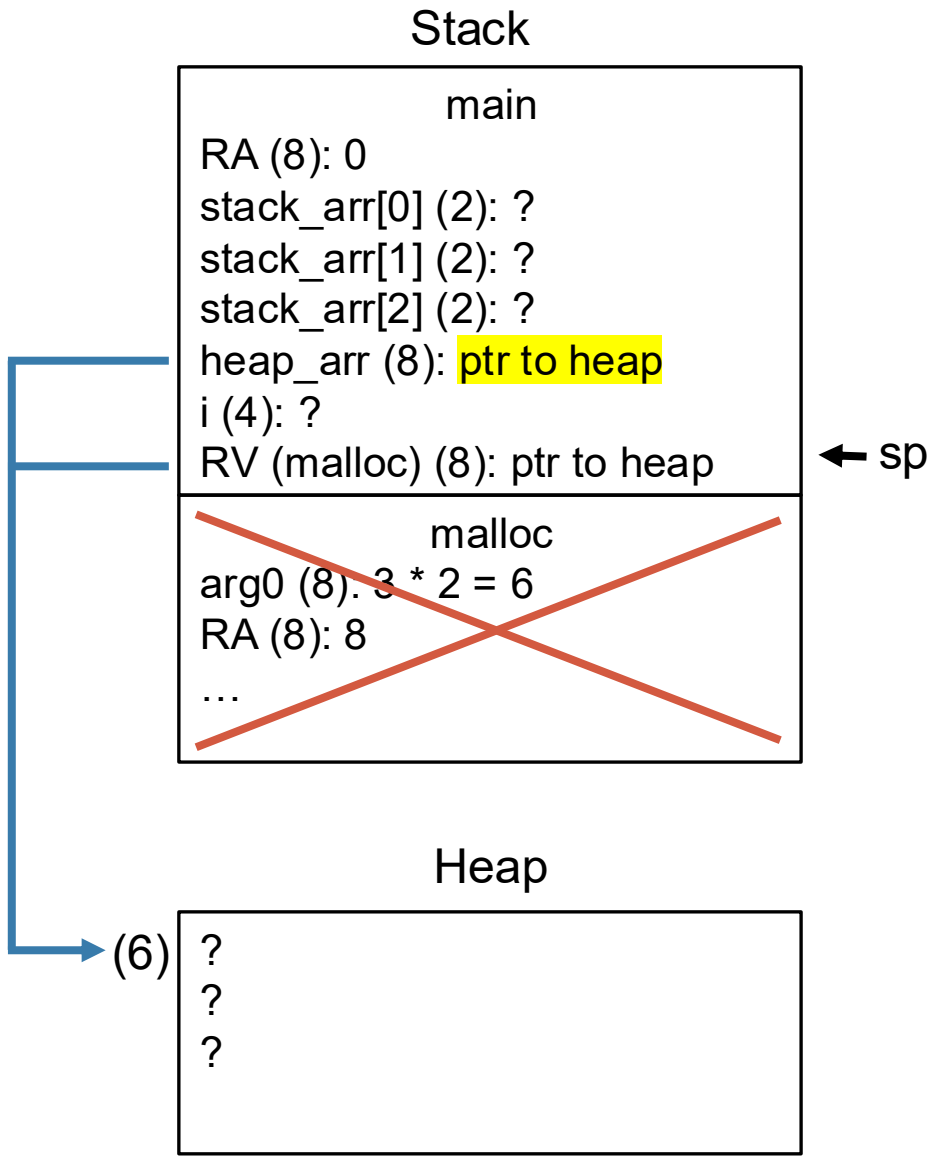
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

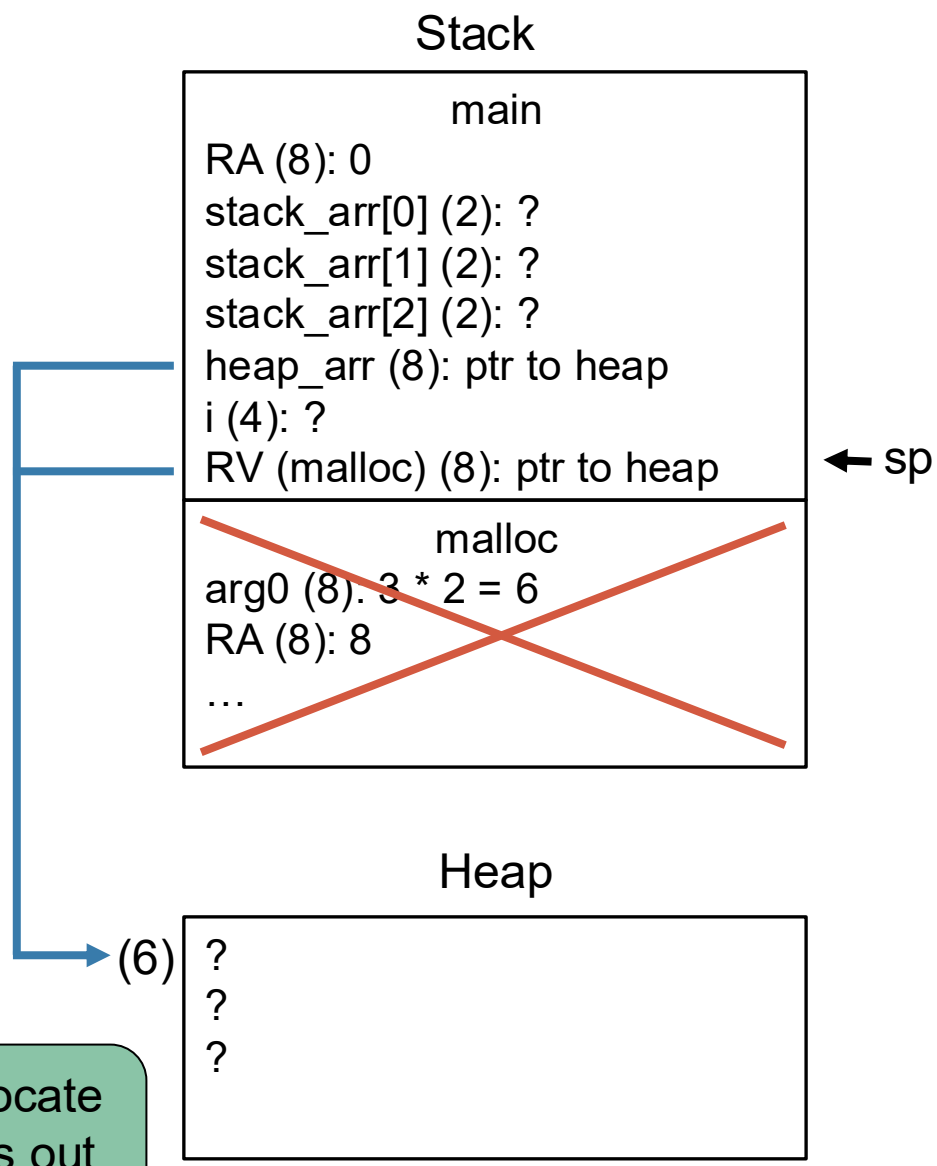
#define ARRAY_SIZE 3

int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

}
```



`malloc` isn't guaranteed to succeed. If the system can't allocate the requested memory (for example, because the system is out of memory or the request is too large) `malloc` will return `NULL`.

High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

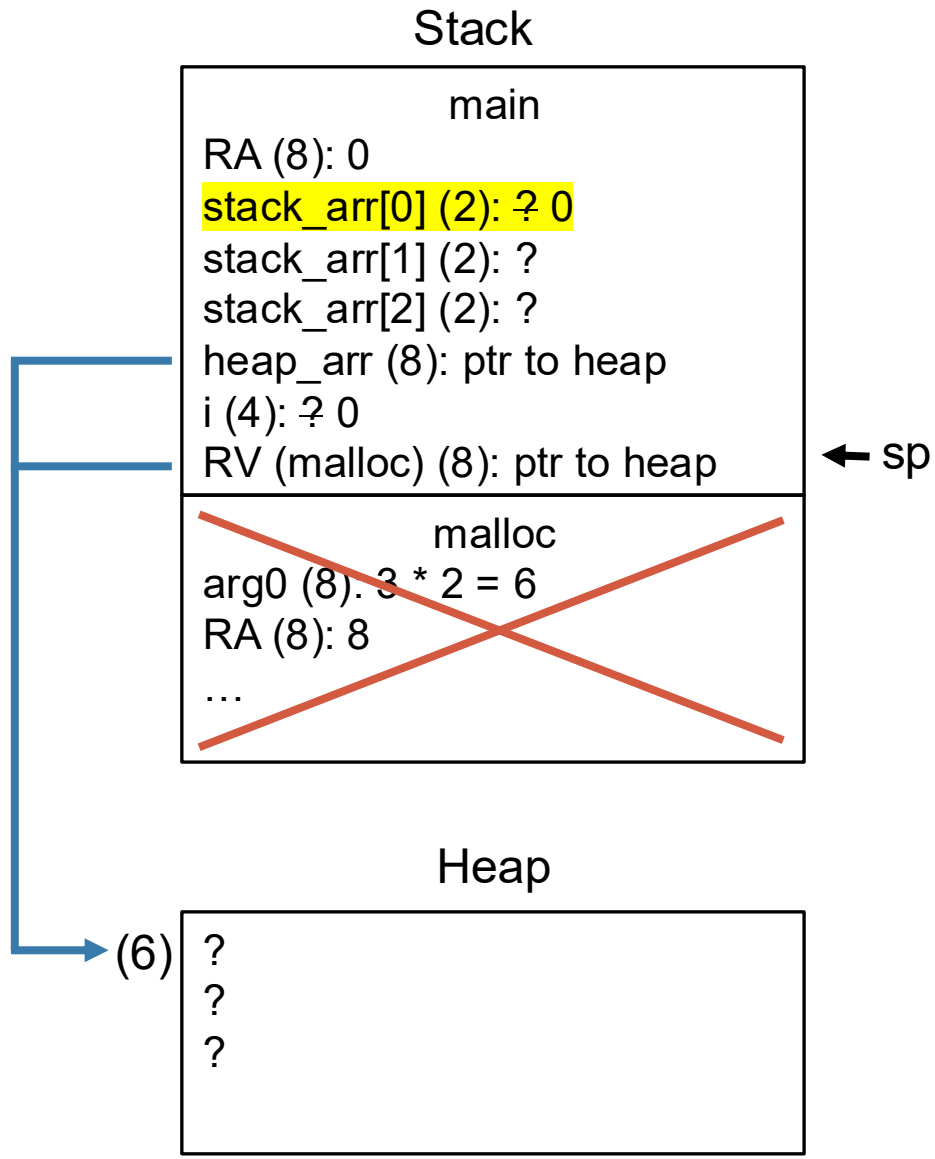
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

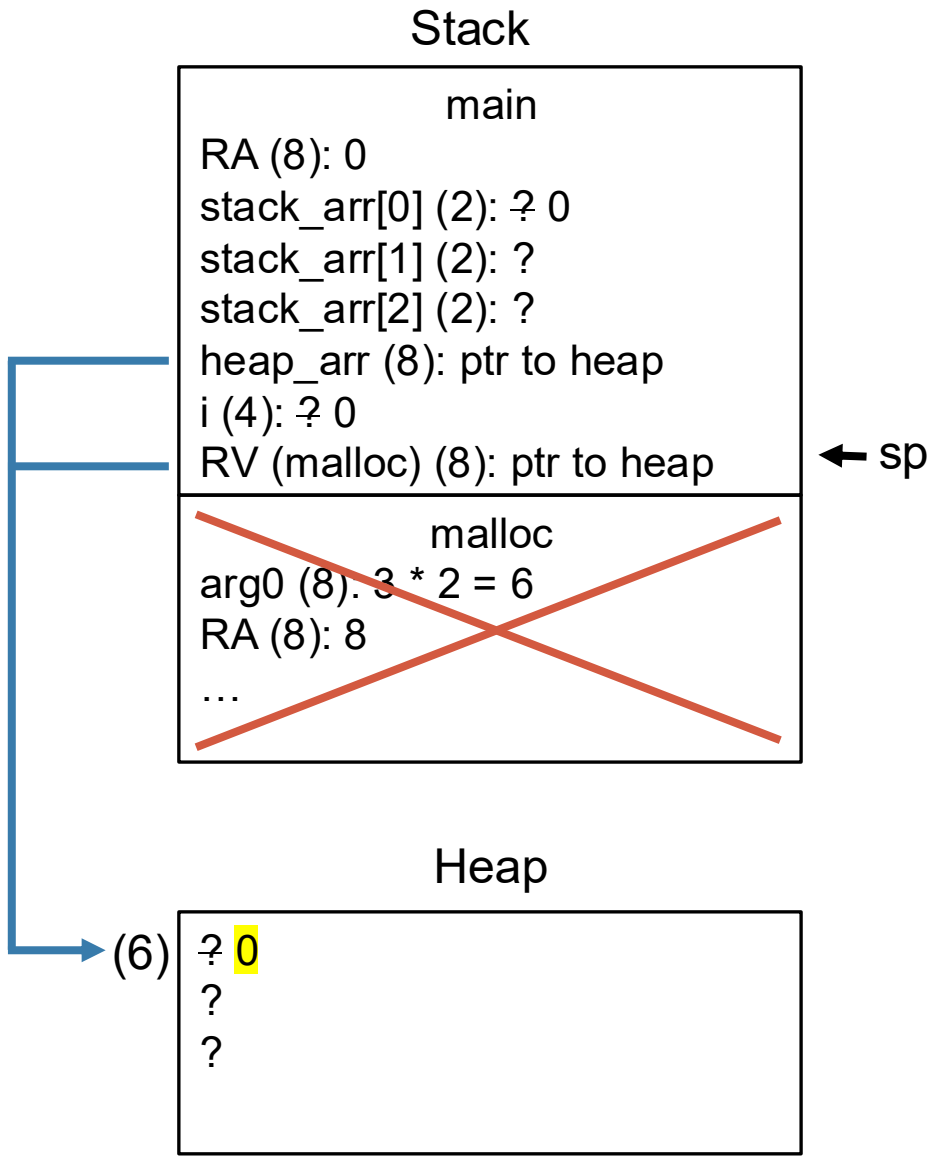
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

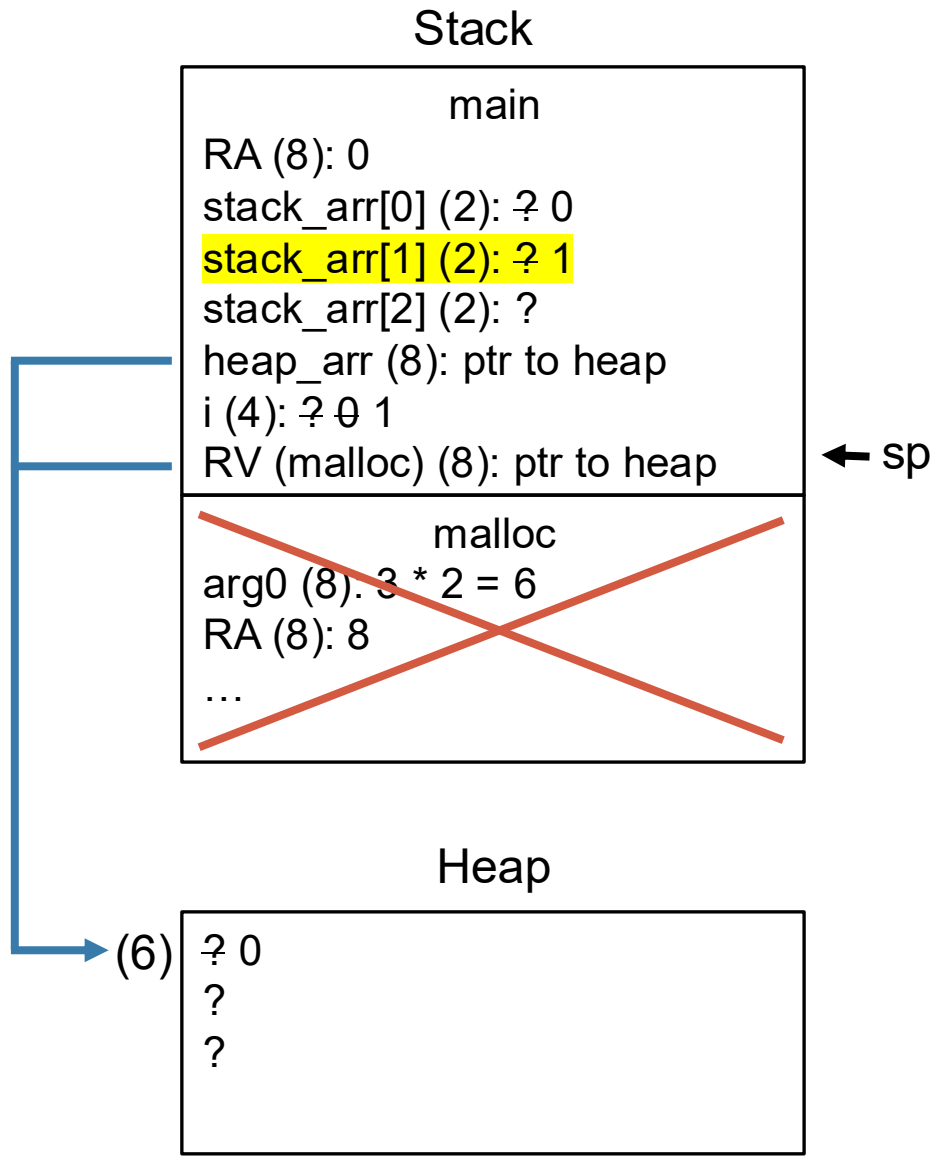
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

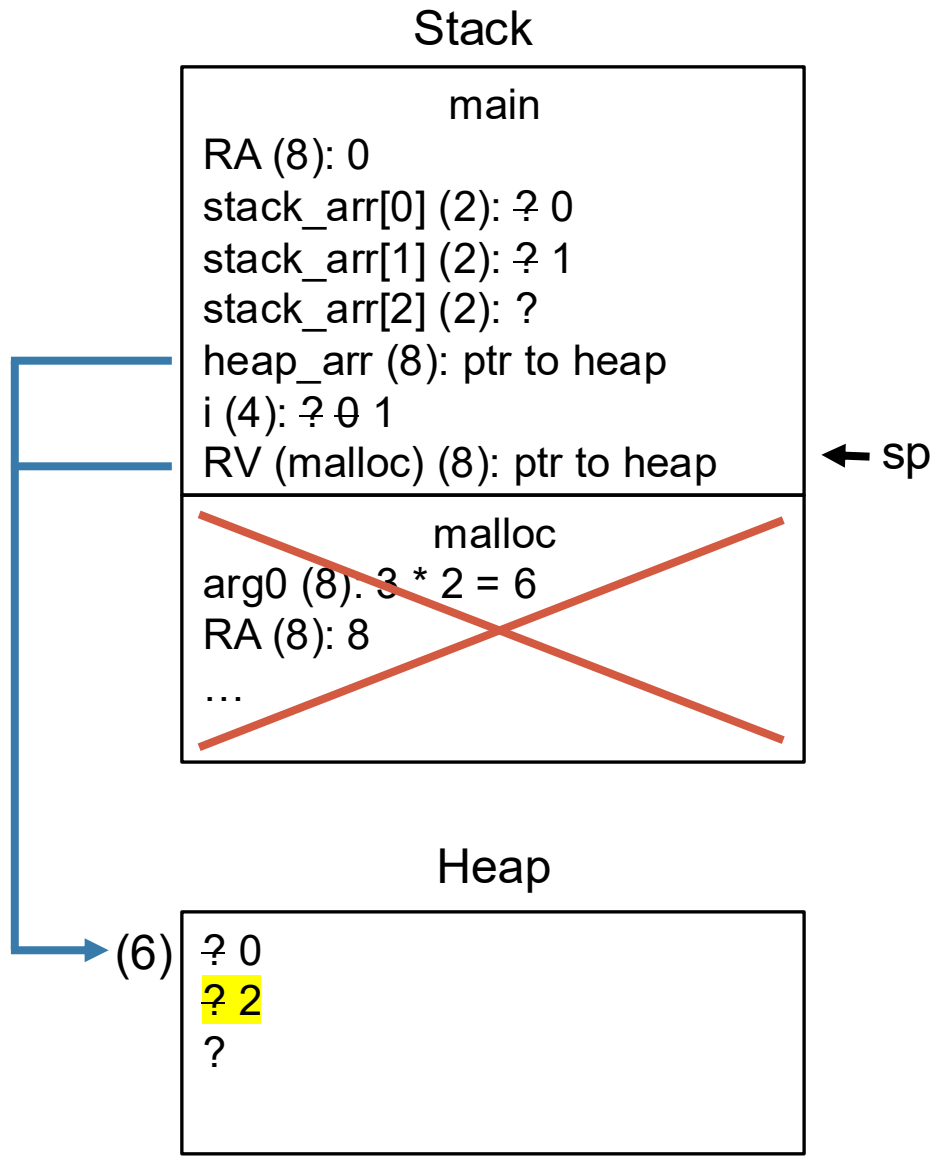
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

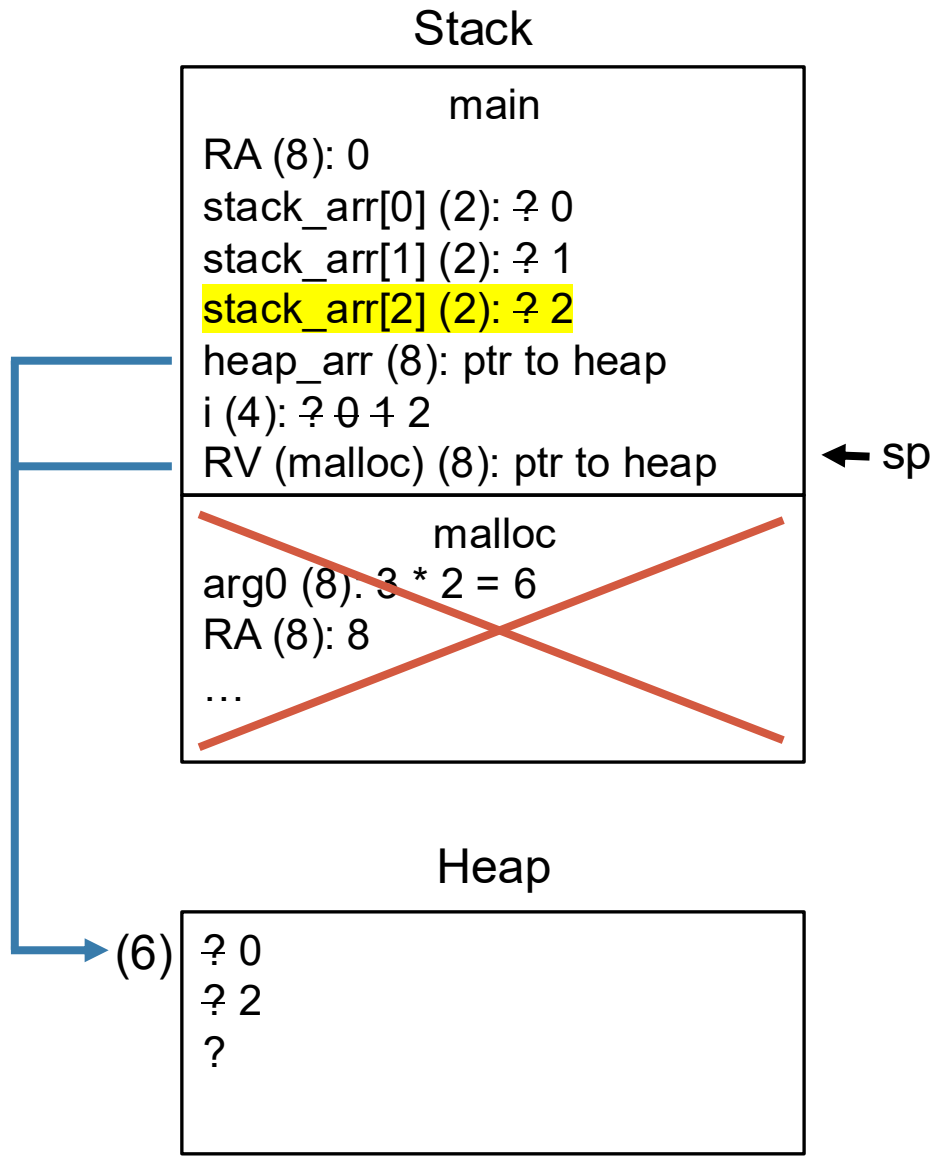
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

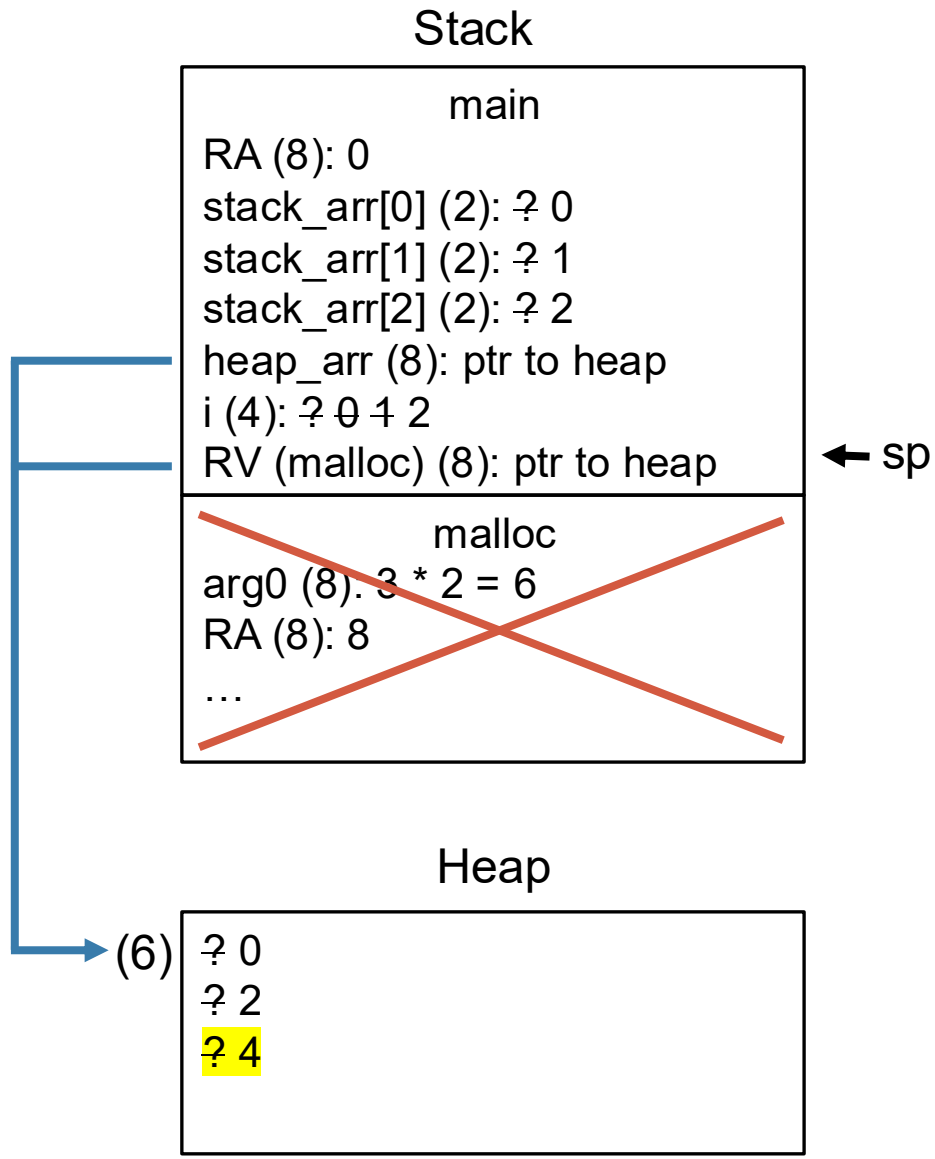
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

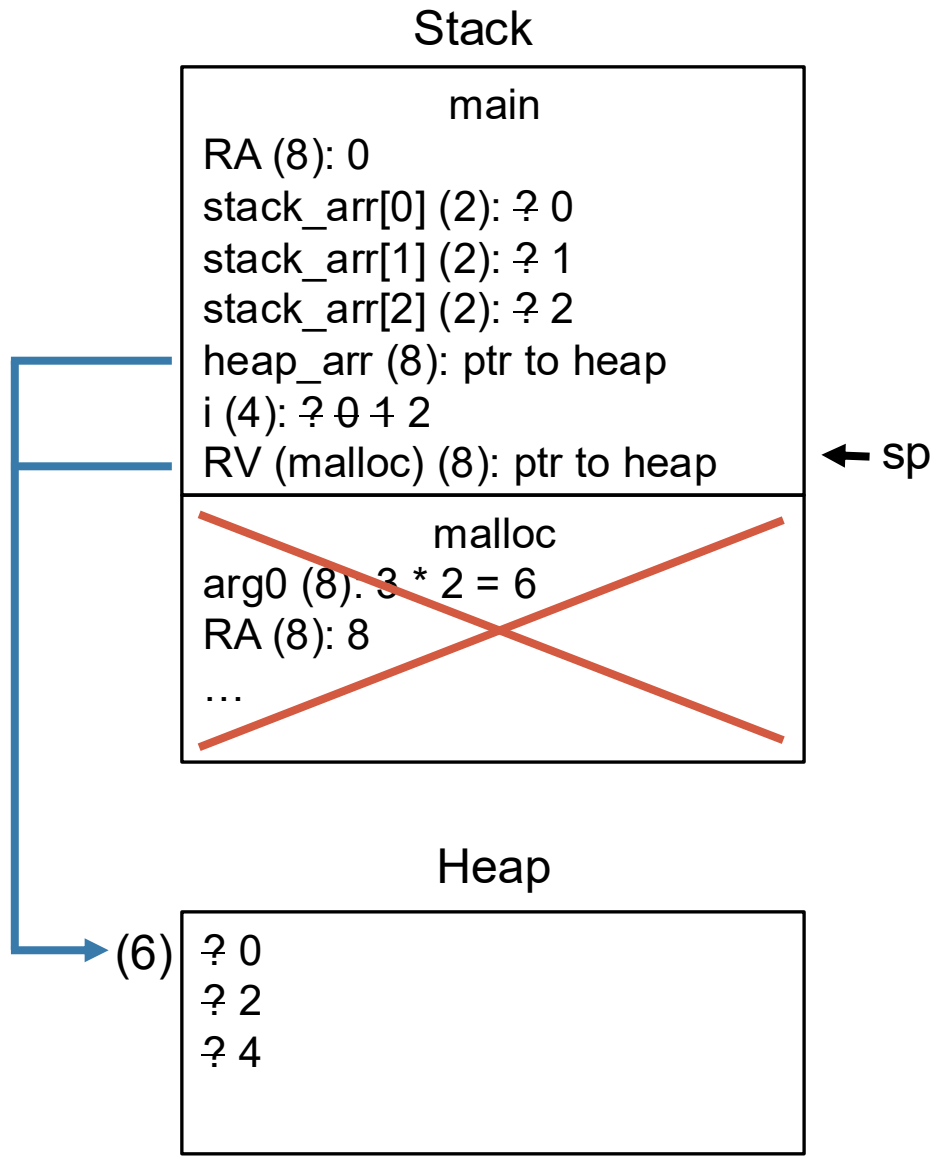
int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

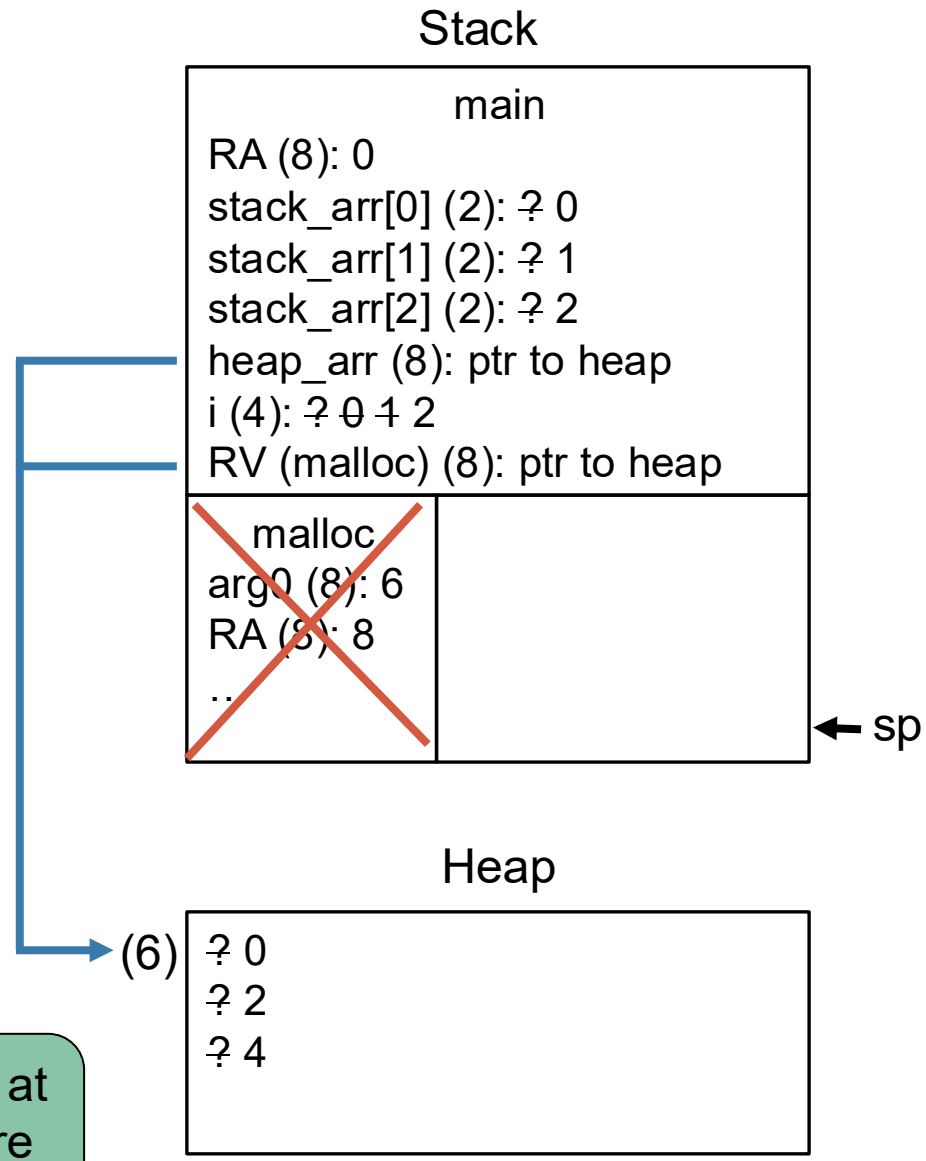
    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```



High-Level Memory Diagram

```
1  #include <ctype.h>
2  #include <stdlib.h>
3
4  #define ARRAY_SIZE 3
5
6  int main() {
7      int16_t stack_arr[ARRAY_SIZE];
8      int16_t* heap_arr =
           (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));
9
10     if (heap_arr == NULL) {
11         printf("Memory allocation failed!\n");
12         return 1;
13     }
14
15
16     for (int32_t i = 0; i < ARRAY_SIZE; i++) {
17         stack_arr[i] = i;
18         heap_arr[i] = i * 2;
19     }
20
21     free(heap_arr);
22     heap_arr = NULL;
23
24     return 0;
25 }
```

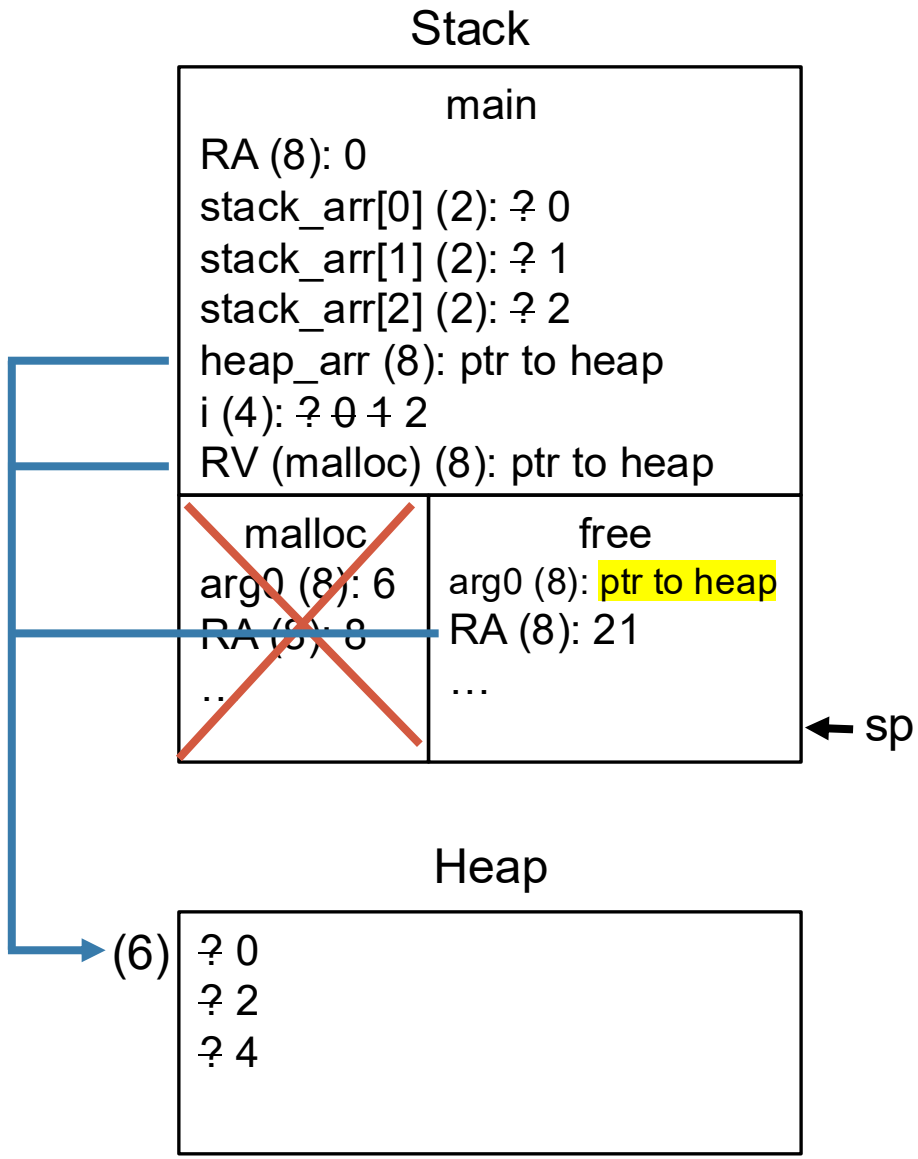


free's frame will begin at the same location where malloc's frame began.

High-Level Memory Diagram

```
1  #include <ctype.h>
2  #include <stdlib.h>
3
4  #define ARRAY_SIZE 3
5
6  int main() {
7      int16_t stack_arr[ARRAY_
8      int16_t* heap_arr =
           (int16_t*)malloc(ARR
9
10     if (heap_arr == NULL) {
11         printf("Memory alloc
12         return 1;
13     }
14
15
16     for (int32_t i = 0; i <
17         stack_arr[i] = i;
18         heap_arr[i] = i * 2;
19     }
20
21     free(heap_arr);
22     heap_arr = NULL;
23
24     return 0;
25 }
```

The `free()` function takes a pointer to the start of the memory block to be released. You don't need to provide the size of the block because the system already knows this from metadata stored in the heap.



High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

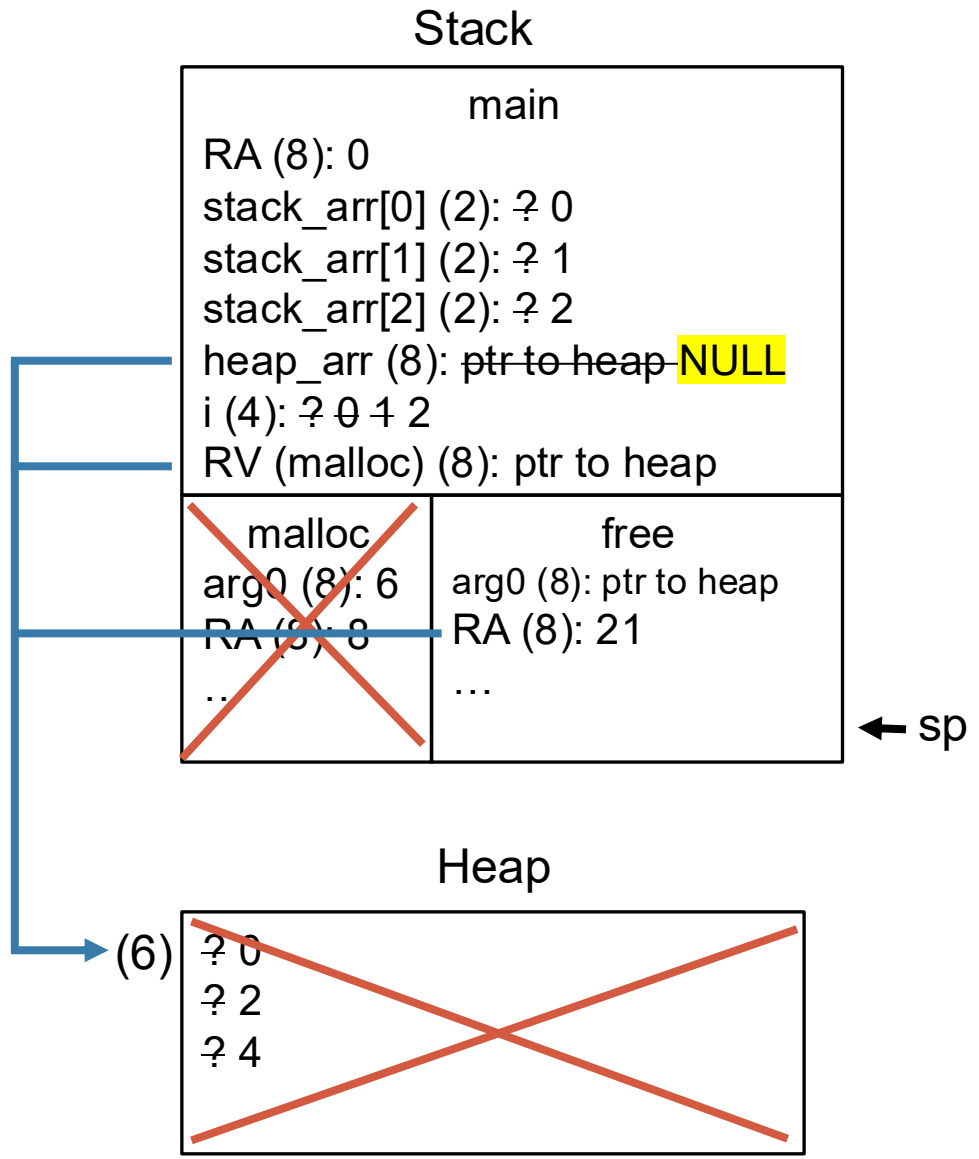
    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```

When `free` executes, it deallocates the block in memory.



High-Level Memory Diagram

```c
#include <ctype.h>
#include <stdlib.h>

#define ARRAY_SIZE 3

int main() {
    int16_t stack_arr[ARRAY_SIZE];
    int16_t* heap_arr =
        (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));

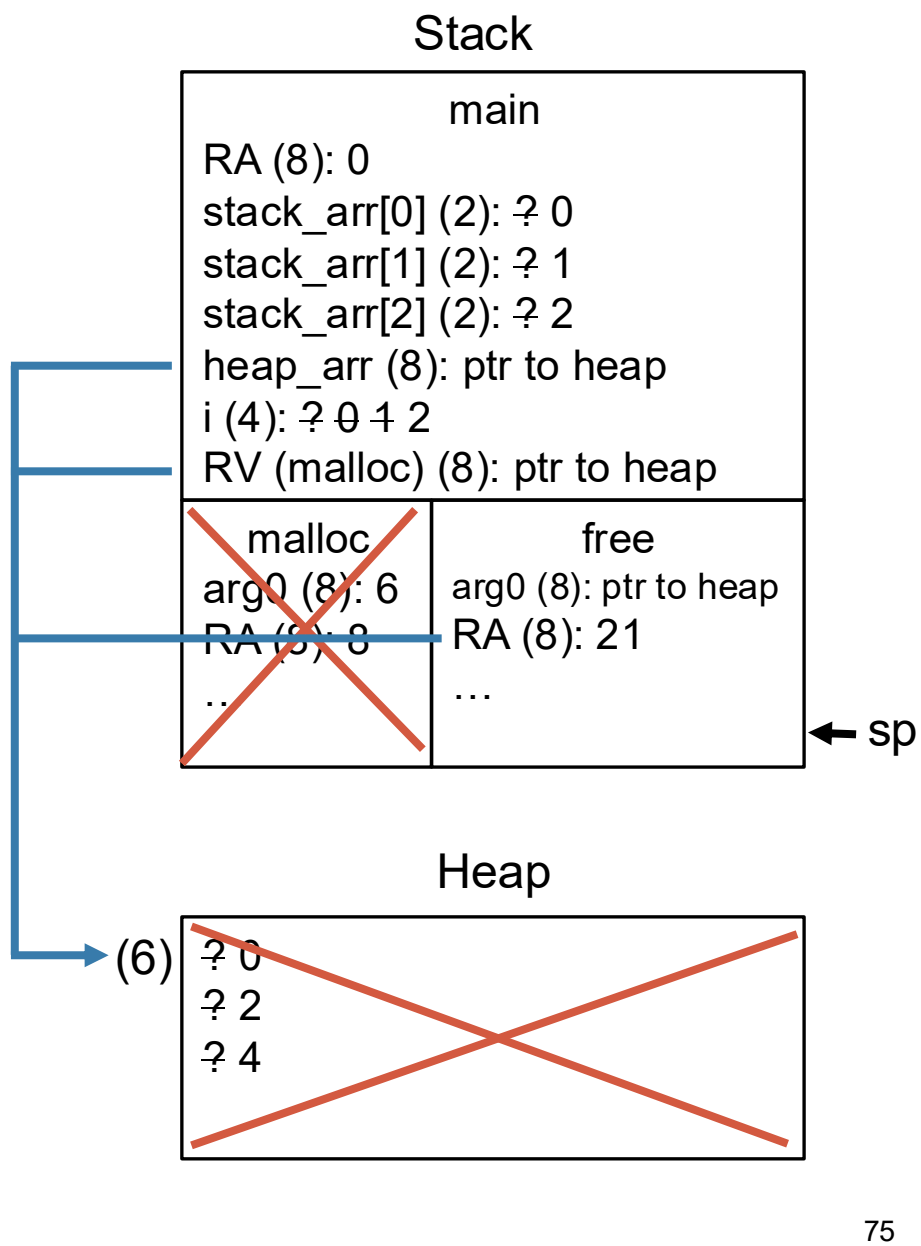
    if (heap_arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }


    for (int32_t i = 0; i < ARRAY_SIZE; i++) {
        stack_arr[i] = i;
        heap_arr[i] = i * 2;
    }

    free(heap_arr);
    heap_arr = NULL;

    return 0;
}
```

Set heap_arr to NULL to avoid a dangling pointer.

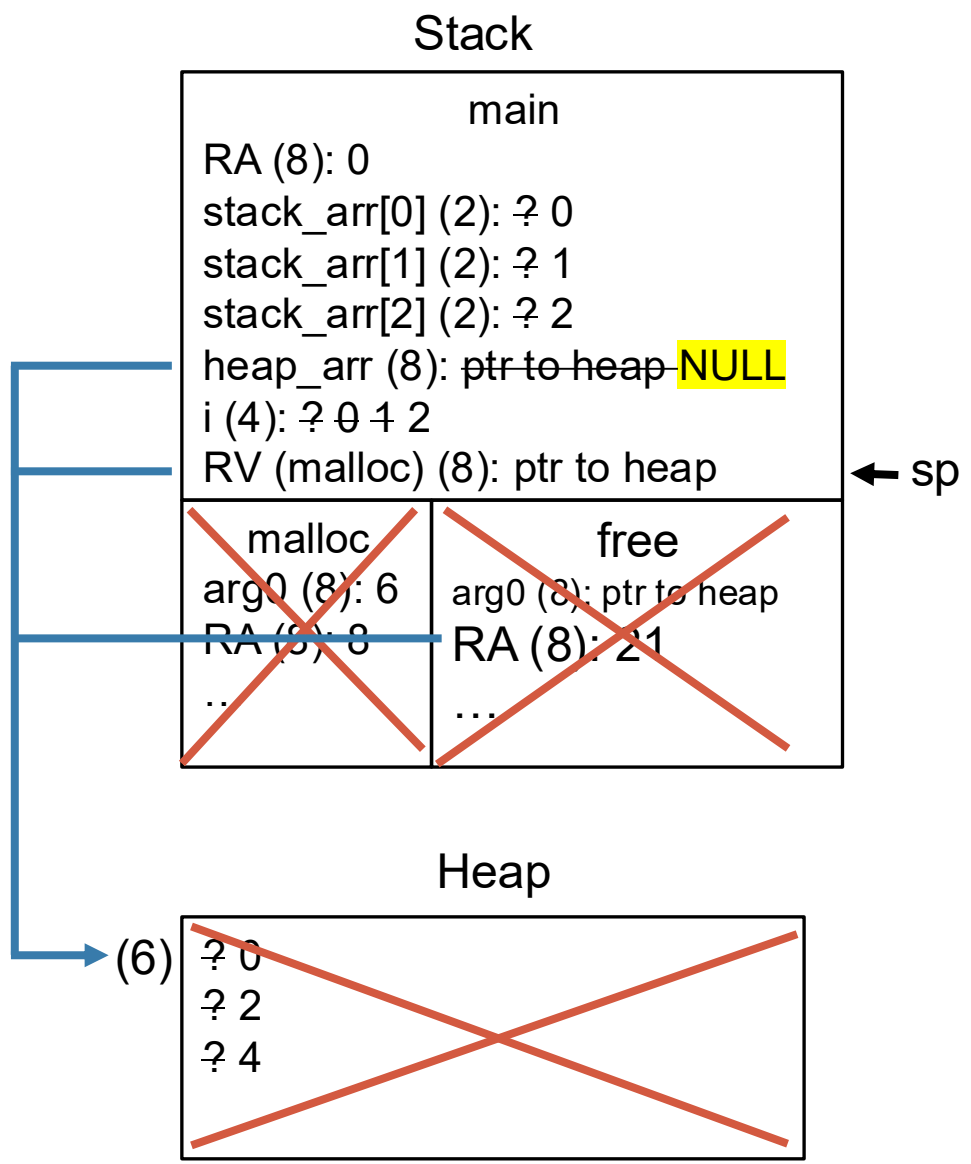


High-Level Memory Diagram

```
heap_arr = NULL;
```

```
1  #include <ctype.h>
2  #include <stdlib.h>
3
4  #define ARRAY_SIZE 3
5
6  int main() {
7      int16_t stack_arr[ARRAY_SIZE];
8      int16_t* heap_arr =
           (int16_t*)malloc(ARRAY_SIZE * sizeof(int16_t));
9
10     if (heap_arr == NULL) {
11         printf("Memory allocation failed!\n");
12         return 1;
13     }
14
15
16     for (int32_t i = 0; i < ARRAY_SIZE; i++) {
17         stack_arr[i] = i;
18         heap_arr[i] = i * 2;
19     }
20
21     free(heap_arr);
22     heap_arr = NULL;
23
24     return 0;
25 }
```

When `free` returns, its frame is deallocated.



Pointer Arithmetic

- For both stack-allocated and heap-allocated arrays, the compiler determines the address of element i using the same basic calculation:

- **Stack-allocated array: stack_arr[i]**
  - The address of element i is:
    - `base address of stack_arr + (i × sizeof(int16_t))`

- **Heap-allocated array: heap_arr[i]**
  - The address of element i is:
    - `base address of heap_arr + (i × sizeof(int16_t))`